DOCTORAL THESIS



Bridging Engineering and Operations: Semantic Interoperability in Industrial Systems of Systems

Oskar Wintercorn

Cyber-Physical Systems

LULEÅ UNIVERSITY OF TECHNOLOGY

Bridging Engineering and Operations: Semantic Interoperability in Industrial Systems of Systems

Oskar Wintercorn

Department of Computer Science, Electrical and Space Engineering
Luleå University of Technology
Luleå, Sweden

Supervisors:

Jan van Deventer & Eric Chiquito

"The unknown future rolls toward us..."

— Sarah Connor, *Terminator 2: Judgment Day*

Abstract

Industrial systems now often operate in interconnected digital environments, where physical assets, software systems, engineering models, and organizational processes must be kept aligned throughout an industrial plant's lifecycle. Although modern industrial technologies enable data exchange and service integration across systems, connectivity alone does not ensure that the systems connected share an interpretation of what they exchange. That gap remains a significant barrier to interoperability, traceability, and the effective reuse of industrial knowledge.

This thesis investigates how ontologies and Semantic Web technologies can provide an explicit semantic layer for industrial systems of systems. It argues that semantic interoperability should not require replacing the systems, engineering tools, and standards already in use. Instead, independently developed representations can be connected through a shared, standards-grounded reference ontology while retaining the distinctions and purposes of their respective domains.

The work develops and applies this approach to industrial systems composed through a service-oriented architecture, using the Arrowhead framework as the concrete environment. The Arrowhead Framework Ontology makes the configured structure of a running deployment available as a machine-interpretable knowledge graph, including which systems provide and consume which services. This representation is aligned with plant-design and lifecycle views through the Industrial Data Ontology, enabling relationships to be traced across operational, engineering, and asset-management contexts. The thesis further examines how different formal representations derived from the same running system support different analyses.

The findings show that explicit semantic representations can complement service-oriented architectures and digital-twin approaches by making relationships between systems, services, assets, and lifecycle information queryable, checkable, and open to inference. They support applications such as diagnosis, traceability, maintenance-oriented analysis, and cross-domain information integration.

Contents

Part I **3**

Chapter 1 – Thesis Introduction 5
1.1 From Service to Serial Number . . . 6
1.2 Scope and Research Questions . . . 9
1.3 Research Methodology . . . 12
1.4 Outline . . . 15

Chapter 2 – Industrial Systems and Service Orientation 17
2.1 Co-Engineering Across the Product Lifecycle . . . 17
2.2 Plant Design and Functional Structure . . . 19
2.3 The Asset Lifecycle Record . . . 20
2.4 Design-Time Documentation, Run-time Drift, and Digital Twins . . . 21
2.5 Cyber-Physical Systems and the Industrial Internet of Things . . . 22
2.6 Systems of Systems . . . 23
2.7 Service-Oriented Architecture . . . 24
2.8 The Arrowhead Framework . . . 24
2.9 Chapter Summary . . . 26

Chapter 3 – Semantic Web Technologies and Ontology Engineering 29
3.1 Levels of Interoperability . . . 29
3.2 Ontologies and Knowledge Graphs . . . 30
3.3 Semantic Web Technologies for Representation and Reasoning . . . 31
3.4 Foundational, Reference, Domain, and Application Ontologies . . . 34
3.5 Ontology Engineering and Competency Questions . . . 36
3.6 Chapter Summary . . . 36

Chapter 4 – Synthesis and Contributions 39
4.1 Development of the Research Program . . . 39
4.2 Run-time Semantic Representation of Industrial Systems of Systems . . . 40
4.3 Reference-Mediated Alignment Across Lifecycle Views . . . 42
4.4 Capabilities Enabled by the Semantic Layer . . . 46
4.5 Chapter Summary . . . 47

Chapter 5 – Reflections and Future Work 49
5.1 Scope and Limitations, in Retrospect . . . 49
5.2 Revisiting the Research Questions . . . 52
5.3 Future Work . . . 53

Chapter 6 – Appended Papers and Research Artifacts 55
6.1 Papers, Research Artifacts, and Individual Contributions 55

References 65

Part II **71**

Paper A 73
1 Introduction 75
2 Key Concepts 76
3 Indistinguishable Twins 77
4 Proof of Concept 79
5 Discussion 82
6 Conclusion 83

Paper B 85
1 Introduction 87
2 Approach 89
3 Discussion 95
4 Conclusion 97

Paper C 101
1 Introduction 103
2 The Approach 105
3 Demonstration 109
4 Discussion 111
5 Conclusion 114

Paper D 119
1 Introduction 121
2 Related Work 123
3 The Arrowhead Framework Ontology 127
4 Replicable Demonstration 132
5 Discussion 140
6 Conclusion 144
7 SPARQL Listings 146

Paper E 155
1 Introduction 158
2 Related Work 159
3 The Alignment Approach 163
4 Methodology 169
5 Demonstration 172
6 Discussion 180
7 Conclusion 183

8 SPARQL and Turtle Listings 186

Paper F 195
1 Introduction 197
2 Related Work 199
3 Dual Representation From Runtime 200
4 Comparison 204
5 Discussion 211
6 Conclusion 215
7 SPARQL and Turtle Listings 216

Acknowledgments

Four years into semantics, and I can still give you five different answers to what a functional location is. Three of them are defensible, which is most of the problem.

Jan, my supervisor and friend, is the reason I ended up here at all. He has an unmatched talent for sending me on side missions, and some of them turned out to be the thesis. Thank you for that, and for the conversations that made most of this work better than it started out. Torbjörn Holm spent three years explaining things to me, often more than once, and built the DEXPI and AP4K models that much of this work stands on. I am grateful for both. Khalid was there through the first two years, when things outside the work were not entirely straightforward, and that mattered more than he probably knows. We also started an experiment we both believed in, which I then let quietly drift into the drawer where such things go. Thank you for the first part, and sorry about the second.

My friends and family asked how it was going often enough to keep me honest and rarely enough to keep me sane. That is harder to get right than it sounds. Minna has heard about ontologies more times than I can count and still has no idea what one is. Four years of explaining should have been enough, so the fault is plainly with the explainer. Tora and My were entirely indifferent to all of it and required walking regardless of deadlines. It helped more than they know.

However, Research does not run on curiosity alone. This work has been supported by the Chips Joint Undertaking Arrowhead fPVN project (Grant Agreement No. 101111977), with funding from the European Union.

Oskar Wintercorn
Luleå, 2026

Use of AI Tools

During the writing of this thesis, ChatGPT, Claude, and Grammarly were used for language support, including grammar correction and suggestions for improving clarity. All such suggestions were critically assessed by the author, who takes full responsibility for the final text. These tools were not used to generate or validate the research results, analysis, or conclusions presented in this thesis.

Acronyms

AAS Asset Administration Shell.

AFO Arrowhead Framework Ontology.

BFO Basic Formal Ontology.

CAD Computer Aided Design.

CPS cyber-physical system.

DEXPI Data Exchange in the Process Industry.

DSRP Design Science Research Process.

FMEA Failure Mode and Effects Analysis.

IDO ISO 23726-3.

IIoT Industrial Internet of Things.

IOF Industrial Ontology Foundry.

IRI Internationalized Resource Identifier.

LCIM Levels of Conceptual Interoperability Model.

OPC UA OPC Unified Architecture.

OWL 2 Web Ontology Language 2.

P&ID Piping and Instrumentation Diagram.

PLC Programmable Logic Controller.

RDF Resource Description Framework.

RDFS RDF Schema.

SHACL Shapes Constraint Language.

SOA Service-oriented architecture.

SoS system of systems.

SPARQL SPARQL Protocol and RDF Query Language.

STEP Standard for the Exchange of Product model data.

SUMO Suggested Upper Merged Ontology.

Part I

Chapter 1

Thesis Introduction

"So much gets lost in the translation."

—Frank Zappa , *interview with Dave Rothman*

The ongoing industrial digitalization has made interoperability a persistent concern [52]. Interoperability is the ability of systems to exchange information and use what they exchange. Traditional industrial automation is structured as a hierarchy of levels, each communicating mainly with adjacent levels [28], which makes responsibilities and interfaces clear but provides no path for information that needs to span levels directly. The Industrial Internet of Things (IIoT) and cyber-physical systems (CPSs) have made such direct interaction both possible and increasingly expected [54]. The IIoT connects industrial sensors, actuators, machines, and software systems, allowing them to share data and functionality across a network [11]. A CPS combines computation, communication, and control with the physical process it observes or affects [36]. CPSs can cooperate as a system of systems (SoS), a collection of independently developed and managed systems that together provide capabilities none could provide alone [39]. Such cooperation is commonly supported through a Service-oriented architecture (SOA), in which a system exposes each capability as a service that other systems can discover and invoke through a defined interface [48]. Together, these developments point toward information moving between systems, engineering disciplines, and organizations that earlier layered architectures, standards, and practices kept more clearly apart.

As organizations adopt modern technologies, they can exchange information more widely, but exchanging it does not by itself mean understanding it. *Syntactic* interoperability lets systems exchange information in compatible formats. *Semantic* interoperability additionally requires that they share an interpretation of what the exchanged information refers to. This remains a challenge in industrial environments, where the same asset is described differently across systems according to domain, author, and lifecycle stage [46].

This is most visible when the same industrial asset must be traced across several engineering views. A process engineer may describe a pump in a Piping and Instrumentation Diagram (P&ID) by its tag, a mechanical engineer may identify it by serial number in a lifecycle record, and a software engineer may refer to it at run-time by the dataflows

it exposes. These descriptions address related aspects of the same installation, but they use different identifiers, vocabularies, tools, and modeling assumptions [21], and nothing records how they map to one another. As long as each description stays within its own domain, the difference may go unnoticed. The problem arises when information must be traced across lifecycle stages, whether for maintenance, diagnosis, or co-engineering. Answering a seemingly simple question, such as which physical pump is associated with a running service, may then require manual cross-referencing between representations that were never designed to meet. Engineers often maintain these connections through personal familiarity with the systems involved, ad hoc conversion tables, or bespoke system integrations. Such solutions may work locally, but they are difficult to keep current. When a pump is replaced, a service is redeployed, or a tag is renumbered, the table or integration that connected them continues to describe the installation as it was, not as it is [27].

The three descriptions of the pump are not duplicate records from which one authoritative version could be selected. They serve different purposes, and each records distinctions the others have no reason to keep. Any one may overlap with the others, but none can be assumed to contain what the others hold. The challenge is therefore not to replace these views with a single universal model, but to make their connections explicit while preserving each domain's semantics. This still requires agreement, but of a narrower kind. Rather than agreeing on one schema for everything each discipline needs to say, the parties need only relate their own vocabulary to a small, published reference vocabulary that already exists and that none of them owns.

By representing information as data connected to other data, rather than as rows in separate tables, the *Semantic Web* provides a foundation for linking engineering, operational, and lifecycle information into a single connected structure that can be queried across its parts. *Ontologies* add a formal layer to this, defining what the terms in such a structure mean so that the meaning is shared rather than local to whoever created the data [23]. Populated with the entities of a particular deployment and the relationships between them, such an ontology yields a *knowledge graph*: a machine-interpretable description that can be queried and reasoned over.

This thesis investigates how semantic descriptions of an industrial SoS can be generated from the running system itself, so that they reflect what is actually deployed rather than what was once specified. Semantic interoperability is then not imposed by replacing existing infrastructure but added as a layer over it, derived at run-time from the systems already in place.

1.1 From Service to Serial Number

Consider a controller that consumes a run-time service named `pumpSpeed`. The name identifies a capability currently available in the running deployment rather than something specified at design time. It does not, however, identify the physical pump that realizes that capability. In the plant-design view, that pump occupies the tagged position `123PU001`, shown in Figure 1.1. In the lifecycle view, the physical unit currently installed at that

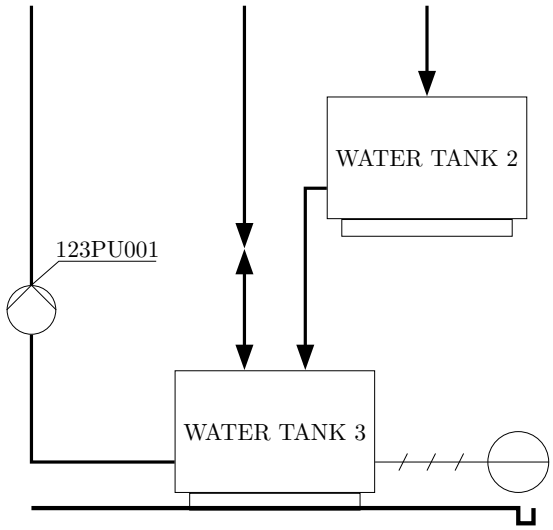


Figure 1.1: An excerpt of the pump station's P&ID. The pump occupies the tagged position `123PU001`, which identifies a functional location in the plant design rather than the physical unit currently installed there.

position is identified instead by the serial number `Pump_24C01213`. Each description is authoritative for its purpose: the run-time view describes what is currently configured, the plant-design view describes the plant's intended functional structure, and the lifecycle view records which physical unit occupies a position over time. None of them alone can answer the full question: which physical pump is associated with the running `pumpSpeed` service?

The challenge is not to select one of these descriptions as authoritative or to merge them into a single model. Instead, this thesis relates them through a shared semantic layer while allowing each view to retain its own vocabulary, purpose, and maintenance process. Figure 1.2 illustrates the resulting path from a run-time service, through the functional position represented in the plant design, to the physical unit recorded in the lifecycle view.

The figure shows the cross-domain path that the query must follow. The run-time asset associated with `pumpSpeed` is linked to the functional location identified by `123PU001`, while the lifecycle view records `Pump_24C01213` as the unit currently installed at that location. With these relationships represented in the aligned graph, the engineer can pose the following cross-domain query:

> *Given the run-time service* ***pumpSpeed****, which functional location in the plant does it concern, and which physical unit is currently installed at that location?*

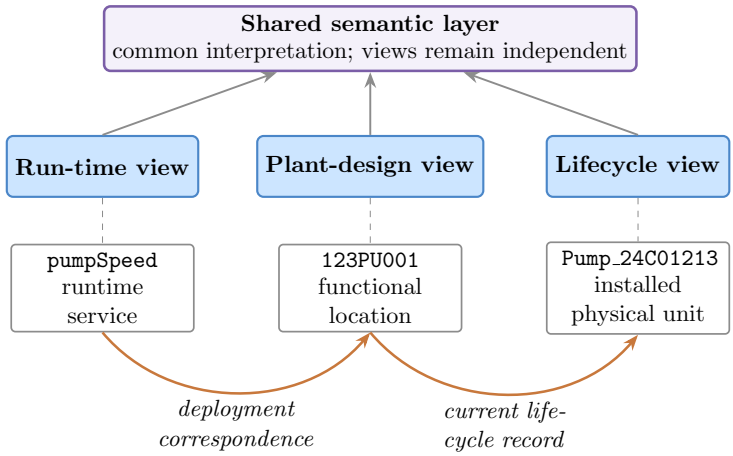


Figure 1.2: A cross-domain trace from a running service to the physical unit currently recorded at its plant position. The semantic layer relates the independently maintained run-time, plant-design, and lifecycle views without replacing them.

It returns the result shown in Table 1.1.

Table 1.1: The answer to the question posed in Section 1.1, obtained from a single cross-domain query.

?svc	?tag	?unit
pumpSpeed	123PU001	Pump_24C01213

The query is short, but its result depends on making the relationships between the three views explicit and machine-interpretable. No source ontology or domain graph is rewritten: the run-time graph remains generated from the running deployment, the P&ID representation remains the plant-design engineer's, and the lifecycle record remains the lifecycle engineer's.

This does not mean that the alignment infers every correspondence from nothing. Two relationships in the trace are deployment-specific facts that must be recorded explicitly. One relates the deployed run-time asset to the functional location it serves, and the other is the lifecycle record's statement of which physical unit occupies that location. Once both are available, however, the trace can also be followed in reverse. Given a serial-numbered unit scheduled for replacement, a maintenance engineer can identify the registered services associated with the position it currently occupies. This result is the endpoint the thesis

aims for, rather than its starting assumption. It requires a run-time representation that describes what is actually deployed, an alignment mechanism that relates independently maintained views without replacing them, and semantic techniques that support querying, reasoning, and validation. The scope and research questions that follow specify how this thesis investigates those requirements.

1.2 Scope and Research Questions

This thesis addresses run-time semantic interoperability in industrial SoS. The Arrowhead framework [16] provides the SOA for the thesis's core contribution. It was selected partly for practical reasons, since the work builds on ongoing project involvement and existing experience with the framework, and partly because its core services expose structured, queryable metadata about a deployment's configuration that a run-time semantic representation can be generated from directly. Most of the appended papers describe, deploy, and reason over a run-time SoS on this architecture. Two of the appended papers apply the same underlying ontology-based approach outside the Arrowhead framework, to a static plant description and to a supply-chain compliance setting. These two papers are included as part of the thesis's development rather than as claims of generalization beyond the Arrowhead framework, and the discussion treats them accordingly.

This thesis makes three contributions, developed principally through Papers D–F and grounded in the motivation and methodological groundwork established by Papers A–C. First, it develops the Arrowhead Framework Ontology (AFO), a Web Ontology Language 2 (OWL 2) ontology that makes the configured state of an Arrowhead-framework-based industrial SoS explicit as a queryable, machine-interpretable knowledge graph, generated directly from a running deployment rather than authored and maintained by hand. Second, it develops an Industrial Data Ontology (IDO)-mediated alignment architecture that connects this run-time representation to a plant-design model expressed in Data Exchange in the Process Industry (DEXPI) and to the Standard for the Exchange of Product model data (STEP)-derived lifecycle model AP4K through modular bridge ontologies, without requiring any source model to be replaced or merged. Third, it demonstrates the reasoning, validation, and representational capabilities this combined run-time and aligned representation enables, including inference, Shapes Constraint Language (SHACL)-based validation, cross-domain traceability, and a comparison with an alternative structural formalism (SysML v2). The AFO, alignment architecture, and bridge ontologies are contributions of this thesis; the domain models they relate are not. This includes STEP AP4K, which Paper E is the first to describe but which was developed by Torbjörn Holm and Alexander Svensson.

Together, the three contributions answer a set of research questions. The first concerns generating a semantic description of a running system. The second concerns connecting that description to the descriptions other disciplines maintain. The third concerns what the connected description makes possible.

RQ1: *How can a practical, reusable semantic description of an industrial SoS be generated from the running system and kept current as the deployment changes?*

At the level exposed by a SOA, interoperability means that independently developed systems can discover, invoke, and exchange data with one another through defined service interfaces. This is a real and necessary property, and service exposure and digital-twin tooling can deliver it directly. It does not by itself mean that the exchanged data carries a shared, machine-interpretable account of what it represents. A service interface can confirm that two systems are connected, and it can let one affect the other's behavior. But it cannot on its own explain why they are connected. Nor can it express that relationship in a form another system or organization can interpret without prior agreement, or support reasoning about dependencies that were never explicitly configured.

RQ1 asks what closing that gap requires, and whether an explicit, standards-based semantic layer can be generated from a running deployment and kept current as that deployment evolves. The rest of the thesis tests what such a layer contributes to a deployment that already achieves service-level interoperability: a machine-interpretable account of what that deployment currently consists of, and the ability to reason over relationships within it that were never explicitly configured.

RQ2: *How can a shared reference ontology mediate alignment between independently developed domain models while preserving domain-specific semantics?*

RQ1 asks what a semantic layer adds to one deployment. RQ2 asks a related but harder question, since most industrial settings involve more than one domain description of the same reality at once. Different domains model that reality differently. Separate communities build their models, and each community makes deliberate choices about how to represent its part of the world. A run-time view of a service-oriented system, a plant design representation, and a lifecycle model each carry their own vocabulary and modeling conventions. Each of these views is correct within its own domain. One common response is to unify them into a single model with one vocabulary and one source of truth. In practice, this is both brittle and counterproductive. Forcing independent models into a common structure distorts the very semantics they were designed to express [47]. It also produces a monolithic artifact that is difficult to maintain as any one domain evolves. A more tractable response is to leave each domain model intact and connect it to a shared reference ontology that acts as a common point of contact. Each model is aligned to the reference independently. Cross-domain reasoning then becomes possible without any model losing its own semantics.

The two approaches also differ in how much mapping work they require as models are added. Relating every model directly to every other requires a number of mappings that grows quadratically with the number of models involved, whereas aligning each model once to a shared reference grows only linearly [44]. Reference-mediated alignment of this kind is an established mechanism in the ontology-alignment literature [47, 32]. What has not been shown is whether it holds when one of the aligned views is not an

authored, stable model but a *dynamic* one, regenerated from a running system each time the deployment changes. RQ2 asks how alignment can be achieved under that condition, in a form that is modular, maintainable, and faithful to each contributing domain, and what role the reference ontology plays in mediating between the domains it connects.

RQ3: *What reasoning, validation, and representational capabilities does a run-time-generated, ontology-based knowledge representation enable, and what does each of these make possible?*

RQ1 and RQ2 concern building and connecting a semantic representation of a running system. RQ3 asks what such a representation is good for. A description that merely restated what the underlying systems already hold would add little, so the value of the semantic layer lies in what can be asked of it, derived from it, checked against it, and traced across it once it exists. These are capabilities that are difficult to achieve with conventional data models.

Four capabilities are of particular interest. Querying and diagnosis operate at the level of the composed deployment rather than the individual interface. Inference derives relationships that were never explicitly configured. Validation checks structural and semantic constraints across system boundaries. Traceability follows a run-time observation to the engineering and lifecycle information needed to explain it. RQ3 asks which of these a run-time-generated, ontology-based representation enables, and what each makes possible that conventional data models do not. It also asks what the choice of formalism itself contributes, which Paper F addresses by comparing the graph-based representation against a structural model generated from the same deployment.

The research questions are addressed cumulatively across the appended papers rather than by isolated studies. Early work establishes the industrial problem and the role of ontology-based knowledge representation, while later work develops the run-time semantic representation, aligns it with engineering and lifecycle views, and evaluates the capabilities that this representation enables. Table 1.2 provides an overview of this relationship.

The mapping identifies the primary contribution of each paper to the research questions, but it should not be read as an exclusive classification. Several papers contribute supporting evidence across more than one question. In particular, Papers D–F form the core integrated contribution of the thesis, while Papers A–C establish the co-engineering context and the semantic-representation and validation capabilities on which that contribution builds. Chapter 4 returns to this mapping and synthesizes what the papers collectively establish.

The questions are limited to semantic interoperability. Where a run-time SoS is involved, the Arrowhead framework is assumed to provide the underlying connectivity between systems. Security is likewise outside the scope. Authentication, authorization, and data protection are critical concerns for any networked industrial system, and they interact with the representation in ways worth investigating, but they are not addressed here.

The work is bounded in three further respects. It covers plant design, lifecycle records,

Table 1.2: Relationship between the appended papers and the thesis research questions.

RQ	Primary papers	Role in the thesis
RQ1	Papers D, E, and F; supported by Papers A and B	Develops a run-time semantic description of an industrial SoS and examines what it contributes beyond service-level interoperability and design-time models.
RQ2	Paper E; conceptually grounded in Paper B	Develops and demonstrates IDO-mediated alignment of run-time, plant-design, and lifecycle representations while preserving their domain-specific semantics.
RQ3	Papers C, D, and F; supported by Paper E	Demonstrates querying and diagnosis, inference, SHACL-based validation, and cross-domain traceability, including FMEA-oriented analysis and a comparison of RDF/OWL 2 and SysML v2 as complementary representation formalisms.

and run-time operation, but neither the definition of requirements at one end of the lifecycle nor decommissioning at the other. Its demonstrators are laboratory-scale and single-organization, so the approach is not evaluated against externally operated industrial deployments or across genuinely independent organizations. And the representations it produces describe configured or declared state rather than independently verified physical reality. Section 5.1 returns to each of these in light of what the completed work demonstrates.

The research methodology described in the following section explains how these contributions were developed and evaluated as a connected design science research program.

1.3 Research Methodology

The work conducted throughout this journey follows the Design Science Research Process (DSRP), a methodology well suited to research that produces and evaluates artifacts such as ontologies, knowledge graphs, and frameworks. Design science research focuses not only on understanding a problem but also on constructing and evaluating solutions, contributing knowledge through the artifacts themselves and the insights gained from building and testing them. This thesis follows the design science research process described by Peffers et al. [49], whose six stages, Identify, Define, Design, Demonstration, Evaluation, and Communication, are illustrated in Figure 1.3 together with the iteration loops that reflect the non-linear nature of the research process.

The process begins with problem identification. This work grew out of discussions with industrial partners and an assessment of how digital transformation is pursued in industrial settings, where systems are heterogeneous, built independently, and rarely replaceable. Mapping the existing landscape made clear that the difficulty lies less in connecting systems than in making the information they exchange usable across their boundaries. From this, the objectives of a solution were defined. Semantic interoperability emerged as the central concern, since the difficulty was not connectivity but shared

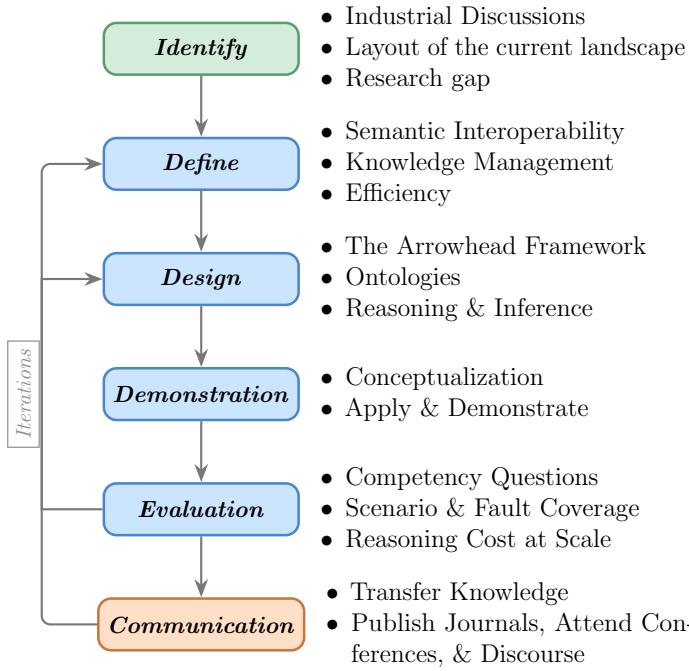


Figure 1.3: The DSRP as applied in this thesis. The iteration loops reflect the cyclic nature of the research, with evaluation outcomes informing subsequent design decisions and communication activities motivating objective redefinition.

meaning, along with managing the knowledge such interoperability makes available and the efficiency with which it can be reached. These objectives set the criteria against which the artifacts developed in this thesis are judged.

The design and development stage produced the artifacts themselves. The Arrowhead Framework provided the service-oriented foundation on which industrial SoS were described, ontologies supplied the formal vocabulary for those descriptions, and reasoning and inference were used to derive knowledge that the underlying systems did not state explicitly. Each artifact was first conceptualized and then applied to a concrete setting, forming the demonstration stage and grounding the work in running systems rather than illustrative examples alone.

Evaluation follows the demonstrations, and the primary instrument throughout is the competency questions (CQs). A competency question states, in the vocabulary of the intended user rather than of the ontology, something the representation must be able to answer if it is to be useful at all. Which systems provide a given service, which consumed dependencies are currently unsatisfied, which physical unit is installed at a given functional location. Formulating these questions before an artifact is built constrains

its scope, and evaluating them afterward against a deployed instance establishes whether the artifact delivers what was intended, not just whether it is internally well-formed.

A competency question is treated as answered when three conditions hold. The corresponding query executes over the instantiated graph after reasoning and returns the expected bindings. It requires no modification to any source model and no separate consultation of the domain-specific models outside the shared vocabulary. Moreover, where the question concerns an inferred relation rather than an asserted one, that relation appears in the reasoner's materialized output and not in the source graphs prior to alignment. The third condition separates what the representation entails from what its sources assert. Entailment is not the same as new knowledge, since much of what an alignment entails re-expresses an asserted fact in a shared vocabulary. That re-expression is nonetheless what allows a single query to cross models whose own vocabularies do not meet.

Three further forms of evidence complement the competency questions. The artifacts are exercised across a range of operating conditions rather than against a single static snapshot, including healthy operation, malformed instance data, degraded operation due to a missing service provider, and recovery after component replacement. The cost of reasoning over the aligned representation is measured directly by generating synthetic instance data at increasing scale and recording materialization time and inferred triple counts. Moreover, when comparing two formal representations, both are generated from the same running deployment and evaluated against a shared set of competency questions, so observed differences reflect the representation paradigm rather than differences in scope, source data, or authorship. Each appended paper states its own competency questions, scoped to the artifact it develops.

The demonstrators are author-developed deployments, so the evidence is proof of concept rather than external validation across independently operated industrial installations. Section 5.1 returns to this and to the other boundaries of the work.

Communication took place through journal and conference publications and through ongoing discourse with both academic and industrial communities, where the knowledge produced by the work is transferred beyond the artifacts themselves.

The process is not linear. Evaluation outcomes regularly fed back into design, prompting revisions to the artifacts and their application. Communication had a similar effect at a higher level, since presenting and discussing the work led to redefining the objectives. The iteration loops in Figure 1.3 reflect these two cycles.

The research questions presented in Section 1.2 are themselves a product of this iteration rather than its starting point. The program began as two largely independent lines of work: one developing run-time, service-oriented interoperability grounded in the Arrowhead framework, and the other developing ontology-based semantic representation and validation techniques. The three research questions took their present form once these lines converged around the AFO, through the same communication-driven redefinition of objectives described above. Section 4.1 traces this convergence and the papers through which it took place.

1.4 Outline

This thesis has two parts. Part I is a comprehensive summary that introduces the concepts on which the work rests and relates the appended papers to the questions they address. Part II contains the papers themselves, reformatted for inclusion, on which the research is based. Part I begins with the present chapter, which establishes the work's context, the research questions, and the methodology followed. A breakdown of the remaining chapters of Part I is presented below.

Chapter 2: Industrial Systems and Service Orientation
This chapter introduces the industrial and technical landscape this thesis addresses. It opens with the co-engineering pressures that motivate the work, then examines two of the three descriptions traced in Section 1.1: the plant design, organized around functional locations identified by stable tags, and the lifecycle record, organized around the physical units that occupy them over time. It then turns to the tendency of design-time documentation to drift from the systems it describes, and to the technical composition that produces the third description: CPSs and the IIoT, SoS, SOA, and the Arrowhead framework as the concrete service-oriented environment used throughout.

Chapter 3: Semantic Web Technologies and Ontology Engineering
This chapter introduces the technologies through which meaning can be made explicit and machine-interpretable: the levels of interoperability that distinguish semantic from technical and syntactic connectivity, ontologies and knowledge graphs, and RDF, OWL 2, SPARQL Protocol and RDF Query Language (SPARQL), and SHACL as the specific technologies used throughout. It then sets out the typology of foundational, reference, domain, and application ontologies used to describe how independently developed models relate to one another, and the practices by which ontologies are built and evaluated, including the competency questions that serve as the primary evaluation instrument in this thesis.

Chapter 4: Synthesis and Contributions
This chapter synthesizes the contributions of the appended papers, organized around the development of the research program and, in turn, around each of the three research questions. It fully develops the concepts specific to this thesis's contribution, including the IDO and the bridge-ontology mechanism that aligns independently developed domain models.

Chapter 5: Reflections and Future Work
This chapter revisits the three research questions in light of the contributions described in Chapter 4, sets out the work's delimitations, reflects on how each held up against what the completed work demonstrates, and outlines directions for future research.

Chapter 6: Appended Papers and Research Artifacts
This chapter documents the publication status, scope, and associated research artifacts of each appended paper, together with the doctoral candidate's individual contribution to each work.

Chapter 2

Industrial Systems and Service Orientation

"If I'm not me, then who the hell am I?"

—Douglas Quaid, *Total Recall*

The pump station in Section 1.1 appeared in three descriptions maintained by three disciplines, related to one another without any of them being altered. This chapter and the next explain what had to be true for that to work.

Several engineering disciplines describe the same asset, each maintaining its own model for its own purposes. Those models are revised separately, on different schedules, and drift from the systems they describe as those systems change. Both are why this thesis derives a semantic representation from the running deployment rather than authoring one. Understanding what has to be related, and why relating it is difficult, starts with how those disciplines work alongside one another.

2.1 Co-Engineering Across the Product Lifecycle

These disciplines rarely work in isolation for long. Co-engineering, also called *concurrent engineering*, is the practice of developing a system's aspects in parallel rather than in sequence, with the disciplines involved working simultaneously and in awareness of one another's constraints [55, 56, 50]. The practice must keep the resulting descriptions consistent across different engineers, tools, and schedules.

The need for co-engineering stems directly from how industrial systems are developed and maintained. A process engineer may describe a plant's functions and connections using a P&ID; a mechanical engineer may maintain a physical model of the same equipment in a Computer Aided Design (CAD) format; a controls or software engineer may develop and revise the logic that monitors and operates it. Each of these descriptions is authoritative within its own discipline, produced by people with the training to interpret it correctly, and each is revised according to a rhythm set by that discipline's own concerns rather than with the others in mind. As long as a change made in one discipline's model is reflected,

sooner or later, in the others, the arrangement can function. However, as the asset is installed, maintained, and further developed, these descriptions may drift since one change in one description is not reflected in the others. Co-engineering spans the practices, tools, and organizational arrangements through which this parallel work is carried out, and keeping the descriptions it produces aligned is a condition of its operation.

Co-engineering is usually discussed as a development-phase concern, where a plant is designed and built from nothing and every description of it is produced fresh. The same alignment is required, and is arguably harder to maintain, in *brownfield* settings, where new equipment, control logic, and software must be introduced into a plant that already exists and cannot simply be replaced as a whole [5]. Replacing a single field instrument, for instance, is not just a matter of installing new hardware. Figure 2.1 follows one such change. The instrument's tagged position is defined in the engineering drawings and does not change. The maintenance records must close out the removed unit and open a record for the installed one. The configuration through which other systems address it must also be revised. Each of these descriptions is maintained separately by the discipline that owns it, and none follows from the others, so the change is complete only once the people responsible for each have brought them into agreement. The disciplines involved must therefore coordinate not only when a system is first designed, but continuously across a lifecycle in which components, connections, and responsibilities keep changing.

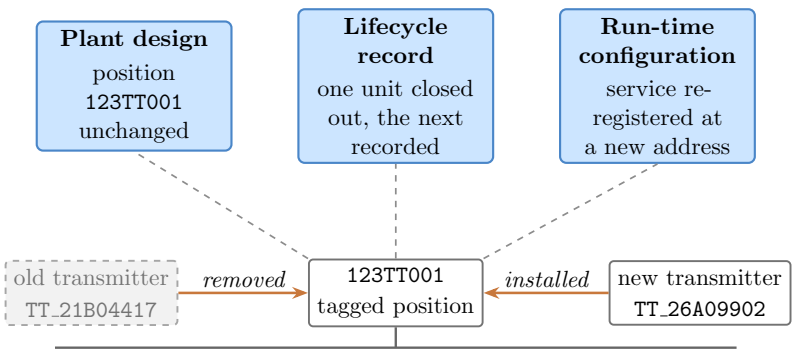


Figure 2.1: Replacing a single transmitter at the tagged position `123TT001`. The position itself is unchanged, so the plant design requires no revision, while the lifecycle record and the run-time configuration must both be brought up to date. None of the three descriptions is derived from the others, and nothing in the change itself records that the new unit now occupies this position.

Coordination becomes harder as the systems being coordinated become more independent, which is the direction SOA pushes industrial systems. A SoS composed at run time from independently developed systems adds parties whose models of the same underlying reality must be kept consistent, without adding mechanisms to keep them consistent [39].

2.2 Plant Design and Functional Structure

A P&ID is the process engineer's authoritative description of what a plant does. Although it looks like a drawing, it is not a geometric picture of the plant. As Figure 1.1 shows, it records which equipment performs which process function, how material flows between those functions through piping, and which instrumentation measures and controls the result [20]. Relative positions on the sheet carry no commitment about physical layout, and a separate CAD model maintained by a different discipline addresses the plant's physical arrangement, as Section 2.1 describes. The P&ID describes function and topology.

Because it is organized around function, the tag `123PU001` in Figure 1.1 does not name a specific pump. It names a position in the plant's structure where a pumping function is required, with enough classification to indicate what kind of function it is. Conventions vary by plant, company, and industry, but a tag of this form typically encodes the area or unit of the plant where the position sits, a code for the class of equipment expected there, and a sequence number that distinguishes it from other positions of the same class. The essential property, for this thesis, is that the tag is stable. The pump installed at `123PU001` may fail, be repaired, be replaced by a different unit of a different make, and be replaced again, without the tag changing at any point. Documentation, control logic, maintenance procedures, and operator practice all address the position rather than the object, precisely because the object is expected to change and the position is not.

The IEC 81346 series formalizes and generalizes this practice beyond the process industry through structuring principles and reference designations [1]. The series treats any technical system as describable through several independent aspects, principally what an object does, what it is, and where it is located, and provides rules for constructing designations that name an object under each aspect separately. Traditional process-industry tags are not reference designations in the strict sense of that series, since they lack its aspect prefixes and its structuring rules. They express the same underlying separation, however, and it is that separation rather than any particular notational convention that matters here. The function performed, the position at which it is performed, and the object performing it are three distinct things, and they may be identified, owned, and revised independently of one another.

Throughout this thesis, a *functional location* means a named, hierarchically arranged place within a plant's structure where a function is performed, and which successive physical units may occupy over the plant's life. The distinction between a functional location and the physical unit occupying it is the hinge on which Section 1.1's cross-domain trace turned, and it is what makes the plant-design and lifecycle views genuinely different rather than redundant. It is also less firmly established in existing vocabularies than its centrality might suggest, a point Chapter 4 returns to when presenting the alignment architecture.

A P&ID is a drawing, however structured the tool that produced it, and what that tool exports is a drawing too. DEXPI makes that content exchangeable by specifying an information model for P&ID data, developed by process-industry operators and software vendors so that it can move between tools without losing structure [2, 20]. The specification

covers the plant breakdown structure, taxonomies and properties for equipment, piping components, and instrumentation, and the topology relating them, with class definitions drawn from the ISO 15926 reference data managed by the POSC Caesar Association. In practice, it is serialized using the Proteus XML schema. For this thesis, what matters is that DEXPI makes the plant-design view a data model rather than a document, and therefore something a semantic representation can relate to rather than merely refer to. The OWL 2 representation of DEXPI used in the appended papers was developed by Torbjörn Holm, Plant&Bytes, and Semantum and is used, not developed, in Paper E. The OWL 2 representation of DEXPI used in the appended papers was produced within the wider research project of which this thesis forms part, and is described further in Paper E. Section 2.3 turns to the third of the three descriptions traced in Section 1.1: the record maintained not of the position, but of the object that occupies it.

2.3 The Asset Lifecycle Record

The lifecycle view, as used in this thesis, is organized around objects rather than positions. Its subject is the particular pump that performs a pumping function: a manufactured item with a maker, model, serial number, purchase date, and history that belong to it rather than to the plant. In Section 1.1, this view answered the final part of the question by identifying the unit at `123PU001` as `Pump_24C01213`. The serial number distinguishes that particular object from interchangeable units and, in the model, remains associated with it as it moves between locations or roles.

Such records accumulate over an object's life. Commissioning, inspections, maintenance interventions, failures and repairs, removal, and eventual disposal can be recorded against the unit itself. In the settings this thesis addresses, that information resides in maintenance or asset-management systems rather than in the engineering models discussed in Section 2.2, and is revised in response to operational events rather than design changes. When a unit is removed, its record is retained rather than overwritten. The value of such a record lies in being able to determine what happened to a given object over time, whether when assessing reliability or when deciding whether a repaired unit should return to service.

The two views meet in a relation that neither model, as scoped here, contains on its own. A physical unit occupies a functional location for a bounded interval of time: it is installed at some point, remains installed for a period, and is eventually removed and possibly replaced by another unit in the same position. The plant-design model does not record this relation, since it describes the position and the function required there without committing to which object currently fills it. The lifecycle record may document installation and removal events, but does not express them as relations to the plant's functional structure. Establishing that `Pump_24C01213` is the unit presently installed at `123PU001` is therefore a statement about a particular deployment at a particular time. It has to be recorded deliberately rather than derived from either description alone. This observation is easy to pass over and turns out to matter considerably. Chapter 4 returns to it when it addresses the limits of what an alignment can and cannot infer.

Recording occupancy over time makes the relation historical rather than merely a statement of the current state. A functional location may be occupied by a succession of units across the plant's life, and a unit may occupy different locations at different times. Asking which unit is installed at a position is therefore always a question about a particular moment, while asking which units have been installed there is a historical question that the same deployment record should answer. Section 2.4 shows why representations that capture only the current state struggle with exactly this.

STEP (ISO 10303) is a family of specifications for representing and exchanging product data throughout the product lifecycle. Its name derives from the Standard for the Exchange of Product model data [6]. Within the relevant STEP models, a product or plant may be described using a breakdown structure whose elements are realized by particular physical items. AP4K adopts this distinction and represents the realization of an element by a physical item as a time-bounded record, following the pattern described in the preceding paragraphs. The appended papers use AP4K, a lifecycle ontology derived from STEP and expressed in OWL 2. It was developed by Torbjörn Holm and Alexander Svensson within the wider research project of which this thesis forms part, and Paper E is the first publication to describe it. As with the plant-design view, what matters here is not the specific encoding but that the lifecycle view is available as a formal model that can be related to others rather than only consulted by hand.

Both descriptions introduced so far are authored and maintained by hand, and both are revised only when someone revises them. Before turning to the last of these, Section 2.4 brings up a problem common to all three.

2.4 Design-Time Documentation, Run-time Drift, and Digital Twins

There is a further complication, and it affects each of the descriptions introduced so far. A P&ID, a CAD model, or a piece of control logic is commonly produced at design time, when a system or plant modification is first conceived, and is thereafter treated as a static artifact, updated only when someone remembers to, has the time to, or is specifically tasked with doing so [9]. The running system it describes, meanwhile, does not stand still. Equipment is replaced, software is reconfigured, and services are added, removed, or rebound to different providers, often for reasons and on a schedule that have little to do with when the corresponding documentation was last touched.

The result is a gap between what a system's documentation says and what the system actually does, commonly called documentation or model drift. This gap tends to widen rather than close over a system's operational life, since every subsequent change is another opportunity for the running system and its description to diverge further, and nothing in how such documentation is typically maintained corrects this on its own. In a dynamically composed, service-oriented SoS, where the providers behind a given service can change at run time, a static document cannot by construction keep pace with a configuration determined dynamically rather than fixed at design time.

The three descriptions do not drift at the same rate or for the same reasons. A plant-design model diverges gradually, as small modifications accumulate faster than anyone updates the drawing. A lifecycle record is event-driven and diverges when an intervention goes unrecorded rather than through neglect over time. A running configuration does not drift in this sense, since it is by definition current. For that reason, there may be no durable record of what it was at any earlier moment.

Digital twins are one widely discussed response to this gap, aiming to keep a digital representation of a system synchronized with its physical counterpart by driving that representation from live operational data rather than from a description authored once and left unchanged [35, 22]. A synchronized twin addresses drift for the one representation it maintains. Relating that representation to the plant-design and lifecycle descriptions that other disciplines maintain is a different task, and one a twin does not perform [40].

Where a digital twin is a representation of an asset kept synchronized with its current state, a *digital thread* is the continuity of information about an asset across the stages of its life and across the disciplines that describe it [3, 40]. The distinction is one of orientation rather than of technology. A twin focuses on depth, maintaining a faithful and current picture of one system. A thread focuses on connection, ensuring that a question raised at one point in a lifecycle can be answered using information recorded elsewhere. The trace followed in Section 1.1 is a thread in exactly this sense. It begins with a service in a running deployment and ends with a serial number recorded in a lifecycle system, passing through a plant-design model that neither of the other two consults directly. A thread of this kind depends on the historical record described in Section 2.3, since a connection that reflects only the present cannot answer a question about what was installed, deployed, or configured at an earlier time.

This thesis develops how it approaches that continuity, and the role a semantically explicit, run-time-derived representation can play in establishing it, once the following chapter introduces the necessary semantic technologies. Section 1.1 has already shown the outcome; the sections below and Chapter 3 supply the two halves of the machinery behind it.

2.5 Cyber-Physical Systems and the Industrial Internet of Things

A CPS embeds computation and communication directly in a physical process, so that the two can no longer be meaningfully separated [37, 41]. Sensors report on the state of the physical environment, controllers and software components interpret that information and decide how to act on it, and actuators carry out that decision by affecting the environment in return. This tight coupling is what distinguishes a CPS from a conventional embedded system. A CPS is not a computer attached to a machine, but a system whose computational behavior is defined by the physical process it operates on and constrained by that process's timing [36]. In industrial settings, the pattern recurs at many scales. It describes an individual field device or Programmable Logic Controller (PLC) acting on its own, and

equally a supervisory control application coordinating many such devices at once.

The IIoT extends this setting beyond the control hierarchy in which such systems have traditionally been confined. CPSs capabilities and information become available to databases, engineering tools, and applications across device, plant, and enterprise boundaries, rather than only to the controllers and supervisory systems directly responsible for them [11]. An asset can therefore exchange information with systems that were never part of its original design and may not share its engineering assumptions.

Connectivity does not make the underlying landscape any more uniform than it was before. A sensor, a controller, and an enterprise information system remain heterogeneous in their protocols, purposes, and lifecycles [41]. The IIoT is therefore not a single system or a shared data model that industrial devices are gradually absorbed into. It is an environment in which independently developed systems must be composed while retaining the distinctions that make each useful.

2.6 Systems of Systems

The individual CPSs introduced above rarely operate in isolation. More often, several independently developed systems are brought together to jointly perform a function that none could achieve on its own [39, 10]. Such a composition is commonly referred to as a SoS. The term carries a claim about how the composing systems relate to one another and to the whole they form, rather than describing size or complexity.

A defining feature of a SoS is that its constituent systems retain meaningful identity and operational independence [39]. Each constituent is typically developed, owned, deployed, and maintained separately, often by different organizations and on different timescales, and each continues to serve purposes beyond its participation in any particular SoS. A supervisory application, a database, and a field controller may together realize a monitoring function that none could realize alone, while each remains useful outside that composition. This is what distinguishes a SoS from a monolithic system, whose components are designed together and are not expected to operate independently of the whole.

Not every SoS exhibits this independence to the same degree. Maier distinguishes SoS by the degree of central management and structural authority exercised over their constituents [39]. At one end are *directed* arrangements, whose constituents are built and operated to serve a single centrally managed purpose; at the other, *virtual* ones, where no central authority or shared purpose exists at all. *Acknowledged* and *collaborative* arrangements occupy the space between. The industrial SoS addressed in this thesis sit closer to the acknowledged end. They have recognized objectives and some coordinating infrastructure, but their constituent systems are neither owned nor centrally designed by a single authority as a directed SoS would be.

A SoS is therefore not designed as a single artifact. It emerges from how its constituents connect and coordinate at a given point in time, and it changes as constituents are added, removed, or reconfigured, without any single party necessarily having full authority over or visibility into the resulting whole. This has direct consequences for how such systems

can be understood. No constituent is designed with full knowledge of the others it may eventually be composed with, so a description of the whole cannot be read off the design of any one of them. It has to be derived from how the constituents are actually related in a given configuration.

Industrial SoS are commonly realized through a SOA, in which systems expose and consume one another's capabilities as services rather than being integrated through bespoke, point-to-point connections.

2.7 Service-Oriented Architecture

SOA is a common way to compose an industrial SoS, and it is the approach adopted throughout this thesis. Rather than integrating systems through connections built for a specific pairing of components, SOA organizes functionality as services: well-defined capabilities that a system exposes for others to use [48, 18]. A service specifies what a capability does and how to invoke it, but not how the provider implements it. Separating the interface from the implementation this way lets the consumer and provider be developed and changed independently, as long as both continue to agree on the interface between them.

That independence, usually called *loose coupling*, is particularly valuable in industrial settings, where equipment and software are rarely replaced or redesigned together. A controller, a piece of analysis software, and a piece of field equipment may each be replaced on its own schedule, driven by obsolescence, upgrades, or failure, without requiring the surrounding systems to change in step. Where point-to-point integration would tie one system's lifecycle to every other system it connects to, a provider can be replaced, upgraded, or reconfigured as long as the service it exposes continues to behave as its consumers expect.

A further consequence is that composition can happen at run-time rather than only at design time. Because a consumer depends on a service's interface rather than on a specific provider, the provider fulfilling that interface at a given moment can be resolved dynamically. A system's behavior is then realized through the services it discovers and invokes, rather than through a fixed, centrally implemented program.

2.8 The Arrowhead Framework

The Arrowhead framework provides a service-oriented environment intended for industrial automation and IIoT settings, and it is the concrete platform on which the SoS discussed throughout this thesis are composed [16, 60]. It organizes cooperating systems within *local clouds*: bounded groups of systems that share a set of core services for discovery, orchestration, and authorization, and within which functionality is exposed and consumed as services. A mission-specific SoS forms within a local cloud by selecting the systems whose services, taken together, realize the required behavior. A local cloud is an acknowledged SoS in the sense of Section 2.6: it has recognized objectives and coordinating infrastructure,

but its constituent systems remain independently owned and developed.

The Arrowhead framework is not the only platform addressing industrial interoperability. OPC Unified Architecture (OPC UA) targets it through an information modeling and client-server communication standard, but is generally applied within a single vendor's device or plant hierarchy rather than as a framework for composing independently developed systems [38]. FIWARE takes a broader, cloud-oriented approach through a shared context-data model and API, aimed more at smart-city and cross-sector application development than at the local, real-time automation setting this thesis addresses [13]. The Asset Administration Shell (AAS), introduced within the Industry 4.0 reference architecture, addresses a related concern from a different angle. Rather than a communication framework for composing services, it standardizes a digital representation for an individual asset, together with servers and registries through which that representation can be hosted and discovered [29]. This thesis does not adopt the AAS as its representational approach, for reasons that depend on how the AAS structures meaning rather than on how it is hosted, and that are taken up in Section 4.2.1. A fuller comparison is given in Paper D.

In this environment, a system is distinct from the device on which it runs. A system is a software entity that provides or consumes services. A device, such as a server, controller, or embedded host, is the physical or computational platform on which one or more systems execute. A single device may host several systems at once, and a system's identity is not tied to a particular device if it is redeployed elsewhere. This distinction matters beyond the framework itself. A dependency between two services is a relation between systems, whereas physical placement, redundancy, and shared failure exposure concern the devices that host them.

Every system shares the same internal structure, illustrated in Figure 2.2. In its most generic form, a system is composed of exactly one husk, its interface to the rest of the local cloud, and one or more unit assets, the hardware or software components that carry out the system's actual function. The husk runs on exactly one host and exposes one or more servers that listen for requests from consumers. A unit asset provides at least one service to other systems and may also consume any number of its own. This pattern allows every system in a local cloud to be described and related to others uniformly, whatever its function.

The framework's core systems handle locating and binding to an appropriate provider [62]. Figure 2.3 shows the three most central of these and how they relate to the application systems that make use of them. A provider publishes its service with the Service Registry, making it discoverable. A consumer, rather than depending on a fixed provider address, requests orchestration, and the Orchestrator resolves a suitable provider at run-time by consulting the registry. Authorization then determines whether the resulting interaction is permitted. The pattern is deliberately decentralized. The core systems mediate only discovery and authorization, and the service interaction itself takes place directly between consumer and provider.

The Arrowhead framework is the operational environment in which the run-time deployments discussed throughout this thesis are described and deployed. Its systems, services, and local clouds are the entities the semantic technologies introduced in the

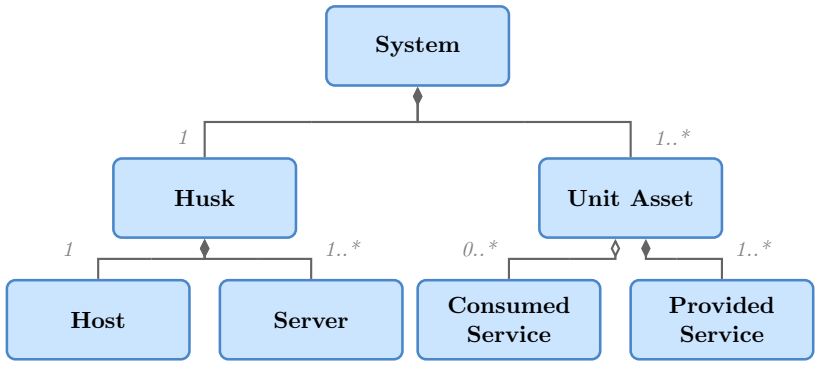


Figure 2.2: The generic composition of an Arrowhead framework system, recurring across every system in a local cloud regardless of its specific function. A system is composed of exactly one husk and one or more unit assets; the husk runs on one host and exposes one or more servers, while a unit asset provides at least one service and may consume any number of others.

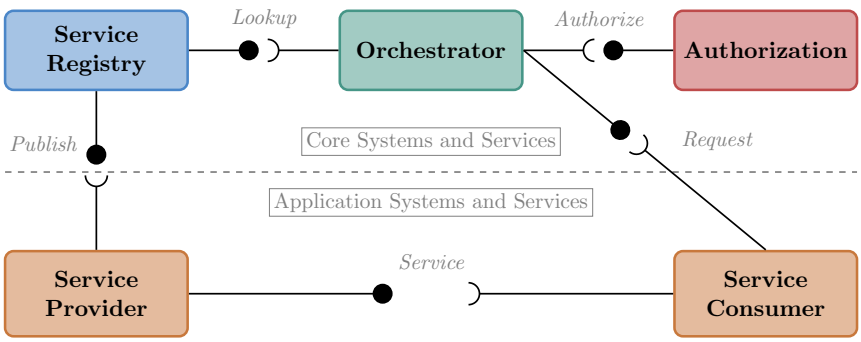


Figure 2.3: Simplified service interaction in an Arrowhead framework local cloud. A provider publishes a service with the Service Registry; a consumer requests orchestration, and the Orchestrator looks up a suitable provider and confirms the interaction is authorized; the service itself is then invoked directly between consumer and provider.

following chapter describe.

2.9 Chapter Summary

Several engineering disciplines describe the same industrial asset, coordinating their work through the practices collectively known as co-engineering. Two of those descriptions were examined directly: the plant design, organized around functional locations identified by stable tags, and the lifecycle record, organized around the physical units that occupy

those locations over time. Coordination between such descriptions is made harder by the tendency of hand-authored, design-time documentation to drift away from the systems it describes. Digital twins offer a partial response to that drift, but do not establish the continuity across disciplines and lifecycle stages that a digital thread requires.

Industrial CPSs sense, decide, and act on physical processes, and the IIoT extends their reach beyond the traditional control hierarchy without making the underlying landscape more uniform. When several such systems are composed to realize a function none could achieve alone, the result is a SoS, most commonly composed in industrial settings through a SOA. The Arrowhead framework provides the concrete environment in which that composition is realized throughout this thesis, organizing systems within local clouds and resolving service interactions dynamically through its core systems.

Composing systems this way multiplies the number of independently maintained descriptions that must stay mutually intelligible, without multiplying the means of keeping them so. The following chapter turns to the semantic technologies through which industrial knowledge can be represented, related, and reasoned over.

Chapter 3

Semantic Web Technologies and Ontology Engineering

"If my answers frighten you, then you should cease asking scary questions."

—Jules Winnfield, *Pulp Fiction*

Two systems can invoke one another successfully, exchange syntactically valid data, and still interpret that data differently, in ways that surface only once the information is put to a use its producer did not anticipate. Connectivity, in other words, says nothing about whether the systems connected agree on what the information they exchange means.

3.1 Levels of Interoperability

Interoperability is better understood as a matter of degree than as a binary property, and it is often described in levels, though frameworks differ in how many they distinguish and how those levels are named. One influential such framework is the Levels of Conceptual Interoperability Model (LCIM), shown in Figure 3.1, which identifies seven levels, ranging from no interoperability at all to full conceptual alignment [59]. Technical, syntactic, and semantic interoperability occupy only its lower tiers, but they are sufficient to make the distinction this thesis relies on.

At Level 1, technical interoperability, a physical or network path exists between two systems and a protocol governs the transfer, so that data sent by one arrives at the other. At Level 2, syntactic interoperability, the exchanged data additionally conforms to a shared format or schema, so that a receiving system can parse it correctly and extract the values it contains. Neither level says anything about whether the extracted values are interpreted correctly. A system can receive a well-formed, correctly parsed value and still have no reliable way of knowing what that value represents or how it relates to a physical asset or process. Nor can it establish whether a similarly named value obtained from a different source describes the same thing. Semantic interoperability, Level 3, is

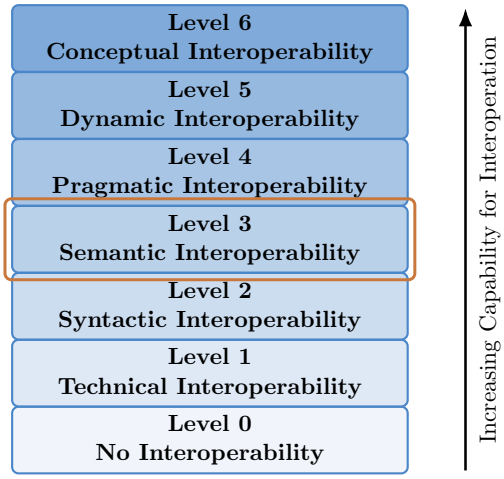


Figure 3.1: The Levels of Conceptual Interoperability Model. The highlighted level marks the semantic interoperability on which this thesis focuses; technical and syntactic interoperability are treated as necessary but insufficient precursors, and the higher pragmatic, dynamic, and conceptual levels fall outside its scope. Adapted from [59].

where this further gap is closed. The exchanged information carries, or can be related to, an explicit account of what it means, shared and machine-interpretable rather than implicit and dependent on a human who happens to know both systems involved.

This thesis focuses on semantic interoperability. It stops short of LCIM's pragmatic, dynamic, and conceptual levels, which concern the alignment of processes, adaptive behavior, and complete conceptual models rather than the representation and machine-interpretation of exchanged information. A system that achieves only technical and syntactic interoperability can exchange information reliably while still failing to make its interpretation available to anything beyond the arrangement it was configured for. Working industrial systems plainly do know what to do with the data they receive, but that knowledge sits in the engineers who configured the binding and in code written against a particular assumption, not in the exchanged information itself. Closing that residual gap is what the remainder of this chapter is concerned with.

3.2 Ontologies and Knowledge Graphs

Closing the gap identified in the preceding section requires more than agreeing, once, that a given piece of information means something in particular. It requires a way of stating what things mean that is explicit enough to be checked, reused across different systems, and interpreted by software rather than only by humans familiar with both sides of an exchange. Ontologies provide exactly this: an ontology is a formal, machine-readable representation of the concepts within a domain and the relationships that hold

between them, intended to make meaning explicit and computable rather than implicit and dependent on a particular person's knowledge [23]. While a conventional data model is typically designed around how information is stored and retrieved efficiently, an ontology is designed around what that information means and how its concepts relate to one another, a different and complementary concern [4].

An ontology on its own defines a vocabulary of classes, properties, and relations, but it does not, by itself, describe any particular situation. That role is filled by populating an ontology with the individuals and the relations between them that describe an actual domain of interest, whether a specific plant, deployment, or supply chain. The result is commonly referred to as a knowledge graph: a body of interconnected facts, expressed using the vocabulary and relations an ontology defines, that can be queried and reasoned over as a whole rather than read record by record. A knowledge graph is consequently only as meaningful as the ontology whose vocabulary it uses. What formalizing that vocabulary buys is consistent interpretation across the systems contributing to the graph, together with the representation, querying, and validation capabilities introduced in the remainder of this chapter.

Not every knowledge graph is built this way. Labeled property graphs, in which attributes are attached directly to nodes and edges and queries are expressed as traversals, are widely used and are often the more convenient choice when a graph is authored and consumed within a single organization. What they do not provide is a standardized account of what the graph's terms mean. Their labels carry whatever interpretation their designers had in mind, their identifiers are local to the store that holds them, and they have no formal notion of entailment. Hence, nothing follows from the data that was not put there explicitly.

This thesis requires all three of the things that absence rules out. The same vocabulary must be usable across models maintained by different disciplines and, in principle, by different organizations, which calls for globally scoped identifiers and terms whose interpretation is published rather than assumed. Relations that hold implicitly have to be derivable rather than asserted case by case. The representation must be checkable against constraints stated declaratively rather than enforced by application logic. RDF and OWL 2 are used throughout this thesis for those reasons, and the technologies that provide these capabilities are the subject of the following section.

3.3 Semantic Web Technologies for Representation and Reasoning

Ontologies and knowledge graphs, as introduced above, describe *what* a machine-interpretable representation of industrial knowledge should achieve. This section introduces the specific technologies used throughout this thesis to represent, reason over, query, and validate such a graph: RDF, OWL 2, SPARQL, and SHACL. Each addresses a distinct concern, and the four are used together rather than interchangeably in the remainder of this thesis.

3.3.1 RDF and Graph-Based Representation

RDF represents information as a set of triples, each consisting of a subject, a predicate, and an object, asserting that some relation holds between two resources or between a resource and a value [14]. A collection of such triples forms a directed, labeled graph, in which resources are nodes and predicates are the edges relating them. This representation suits industrial knowledge particularly well, since relevant facts about a system, an asset, or a process are rarely confined to a single record or table, and RDF lets them connect without first flattening them into a single common schema.

A set of triples considered on its own carries no record of where its assertions came from. Two statements about the same resource are indistinguishable in a single merged graph, whether they originated in the same source or in models maintained by different disciplines years apart, and once merged, there is no way to ask a question of one source rather than another. RDF therefore also defines the notion of a dataset, comprising a default graph together with any number of named graphs, each identified by an Internationalized Resource Identifier (IRI) and holding its own set of triples [14]. SPARQL can then scope a query to a particular named graph, or query across several, so that assertions can be kept separate without being kept apart.

This matters throughout the remainder of this thesis. Keeping each independently maintained model in its own named graph preserves the distinction between what a plant-design model asserts, what a lifecycle record asserts, and what a running deployment observed, while still allowing a single query to follow relations across all three. It also makes it possible to represent the same subject at different points in time by recording successive states as separate named graphs rather than by overwriting a single one, which is what allows the historical questions raised in Section 2.3 to be asked of the representation at all.

RDF does not, by itself, impose any particular schema on the assertions it represents; a triple can be published using any predicate its author chooses, without thereby specifying what that predicate means, which entities may sensibly appear as its subject or object, or what follows from asserting it. RDF Schema (RDFS) extends RDF with a minimal vocabulary for classes and properties [12]. However, its expressiveness is limited to lightweight classification and falls short of the axioms and logical inference this thesis relies on, which OWL 2 provides instead.

3.3.2 OWL and Logical Inference

OWL 2 supplies the formal vocabulary and semantics through which RDF assertions are given a precise, machine-interpretable meaning [43]. An OWL 2 ontology defines classes, properties, and individuals, together with axioms stating how they may relate: class hierarchies, property characteristics such as transitivity, domain and range restrictions, disjointness, and cardinality constraints, among others.

These axioms allow a reasoner to derive facts that were never explicitly asserted but follow logically from the ontology and the available data. If a relation is declared transitive, for instance, and an asset is asserted to be installed within a location that is itself installed

within a larger one, a reasoner can infer that the asset is installed within the larger location as well, without that fact ever having been stated directly. OWL 2 reasoning follows an open-world assumption [51, 43]: the absence of a statement from a graph is ordinarily treated as unknown rather than false, which is appropriate when information is incomplete, distributed across sources, or acquired incrementally, as is typically the case in industrial settings. An open-world assumption is not the same as tolerance for inconsistency, however; a graph may lack information without being inconsistent, whereas an inconsistency arises specifically when the asserted and inferred statements cannot all be true under the ontology's axioms.

Expressive power in OWL 2 is not free. Reasoning over the full language is computationally expensive in the general case, and applications rarely need all of it, so implementations commonly reason over a restricted subset instead. OWL 2 RL is the relevant such subset here, its axioms being expressible as inference rules that a rule engine can apply directly [43]. Lighter rule sets predating it, such as OWL 2-Horst, remain in common use in triple stores and cover much of the same ground more cheaply [58], but not all of it. OWL-Horst, for instance, does not support the property chains that OWL 2 introduced.

Implementations also differ in how they compute those entailments. A materializing store applies its rule set when data is loaded, computing the entailed triples once and storing them alongside the asserted ones, so that subsequent queries become ordinary pattern matching over an enlarged graph. The alternative is to leave the graph as asserted and account for entailments when evaluating a query. Materialization front-loads the cost, which suits a representation that is generated, loaded, and then queried repeatedly. However, it makes the choice of rule set consequential in two respects: the rule set determines both what can be inferred and what the inference costs. Chapter 4 returns to this when the reasoning requirements of the alignment developed in this thesis are considered.

3.3.3 SPARQL Querying

SPARQL is the standard query language for RDF graphs [26]. Its central mechanism is graph-pattern matching: a query specifies a pattern of triples, possibly involving both asserted and inferred facts, and the query processor returns every set of resources in the graph that satisfies it. This lets a query follow relations across originally independent sources of information, without requiring the person posing the query to know how any individual source stores its data internally.

SPARQL supports several forms of operation, including `SELECT` queries, which return variable bindings; `ASK` queries, which test whether a pattern has any solution; `CONSTRUCT` queries, which build a new graph from a query result; and update operations, which add, modify, or remove triples. The ability to query a graph in this way does not remove the need for a well-designed underlying vocabulary: a SPARQL query is only as meaningful as the ontology terms it relies on, which is precisely the role OWL 2 plays in the combination of technologies introduced in this section.

3.3.4 SHACL and Validation

SHACL complements OWL 2 reasoning by validating whether a supplied RDF graph satisfies a set of declared constraints, rather than deriving new facts from it [34]. A SHACL shape states an expectation about a class of resources, for example that every instance of a given class must have at least one value for a particular property, or that a property's value must be of a particular datatype. Validating a graph against such a shape produces a report identifying which nodes, if any, fail to meet the stated expectations.

OWL 2 and SHACL therefore answer different questions about the same graph. OWL 2 asks what follows from the asserted knowledge and the ontology's axioms, under an open-world assumption in which missing information is treated as unknown rather than false. SHACL asks a different question: whether the graph in front of it satisfies a set of stated requirements. It evaluates the data as given, so a required value that is absent is reported as a violation rather than left open as something that might yet be true. The two are consequently complementary rather than competing: inference enriches a graph with consequences that were not explicitly stated, while validation identifies when the information available, asserted or inferred, is nonetheless incomplete or unsuitable for a particular purpose.

3.4 Foundational, Reference, Domain, and Application Ontologies

The technologies introduced in the preceding section make it possible to represent, reason over, query, and validate a body of industrial knowledge. However, they do not by themselves determine which concepts should be shared across independently developed ontologies, or how ontologies developed for different purposes ought to relate to one another. Ontologies are commonly distinguished according to the generality and intended reuse of the concepts they represent. It is useful to introduce that distinction here, since it is what makes it possible to talk coherently about relating ontologies built for different disciplines later in this thesis [25, 61]. Figure 3.2 shows the four layers commonly distinguished on this basis, each specializing the one above it.

A *foundational ontology* describes highly general categories intended to apply across domains, such as objects, processes, qualities, and relations, providing a disciplined account of the basic kinds of entities that more specific ontologies may build on. Basic Formal Ontology (BFO) and Suggested Upper Merged Ontology (SUMO) are widely used examples [7, 45]. Foundational ontologies are useful precisely when independently developed domain models need to be related through compatible high-level distinctions, but they are too abstract to describe the concepts any specific application actually needs.

A *reference ontology* occupies a more domain-oriented position. It supplies a stable, reusable vocabulary for concepts that recur across a broad field of application, retaining enough abstraction that more specialized models can relate to it without replacing the detailed vocabulary any one discipline relies on. The IDO, standardization process under ISO 23726-3, is one such ontology for industrial assets and processes, as are those grounded

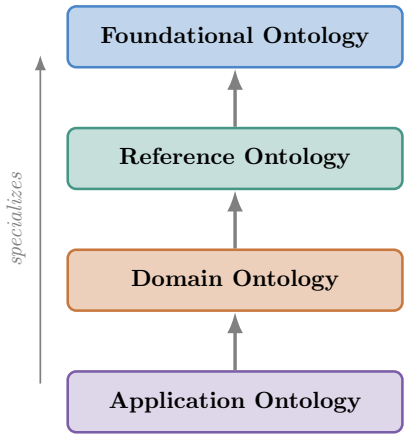


Figure 3.2: Ontology layers by generality and intended reuse. Arrows represent specialization: a reference ontology specializes a foundational ontology, a domain ontology specializes a reference ontology, and an application ontology specializes one or more domain ontologies. Adapted from Fig. 4 of [25].

in ISO 15926 and the Industrial Ontology Foundry (IOF) Core ontology [30, 8, 17]. Their purpose is to provide shared semantic commitments through which independently developed vocabularies can be related, not to serve as a single authoritative model in their own right.

A *domain ontology* represents the concepts specific to a particular area of practice: the equipment, piping, and instrumentation distinguished in plant design, or the sensors and observations distinguished in a monitoring application. SSN/SOSA for sensing, OntoCAPE for chemical process engineering, and SAREF for connected devices are examples [31, 42, 15]. Such distinctions are essential within their own domain and should not be discarded because they do not recur elsewhere. This is where the detailed vocabulary a discipline actually works with is defined.

An *application ontology* is narrower still, scoped to a defined task, system, or deployment rather than to a domain in general, and populated with the individuals and relations relevant to that setting. A description of one particular pump station, of the kind traced in Section 1.1, sits at this level. Such an ontology will commonly draw on domain or reference vocabularies rather than define everything it needs from scratch, but what distinguishes it is the scope it is built for.

These four layers are not always cleanly separable in practice. The same ontology may be reused at more than one level, and its classification can depend as much on how it is used as on how it was designed. The term *upper ontology* is also widely used in this area, though inconsistently: it is applied both to foundational ontologies such as SUMO [45] and to reference ontologies such as those grounded in ISO 15926 [8], and the appended papers use it in the latter sense. The four-layer distinction is nonetheless useful

going forward, since it provides the vocabulary needed to describe how the independently developed ontologies used throughout this thesis relate to one another, and at what level each operates.

3.5 Ontology Engineering and Competency Questions

An ontology is a designed artifact rather than a discovered one. Building it involves deciding what portion of a domain to represent, at what granularity, which distinctions matter enough to be given their own terms, and, equally, what to leave out. Two ontologies of the same subject matter, built by different people for different purposes, may differ substantially and both be defensible, because they encode different decisions about what their users need to say.

This makes scope the central difficulty. Nothing internal to an ontology indicates when it is finished, since any domain admits further detail and any set of terms can be refined further. Judged on internal criteria alone, an ontology is complete when its author stops working on it. Judged against its intended use, it is complete when it can answer the questions its users need answered. Ontology engineering has settled on the second of these.

Competency questions are the standard means of making that criterion concrete [24]. A competency question states, in the vocabulary of the intended user rather than of the ontology, something the representation must be able to answer if it is to serve its purpose. Formulated before construction begins, such questions bound the design, since a distinction that no competency question depends on need not be represented at all. Retained afterward, the same questions become the basis on which the finished artifact is judged. This matters because the two properties an ontology might be checked for are not the same thing. A reasoner can establish that an ontology is logically consistent, and consistency is necessary, but an ontology can be entirely consistent and still fail to support anything its users need to do with it.

A second principle running through ontology engineering practice is reuse. Established vocabularies are, wherever they are suitable, to be adopted rather than replaced by newly authored equivalents, which is much of what the layering described in Section 3.4 exists to support. The reason is not only economy of effort. A term taken from a widely adopted reference ontology carries an interpretation that others already share and already build against, whereas a newly coined term carries only the interpretation its author intended, however carefully that intention was documented. Reuse is therefore a means of inheriting agreement, and it is agreement, rather than formalization as such, that semantic interoperability ultimately depends on.

3.6 Chapter Summary

This chapter has introduced the semantic technologies through which technical connectivity without shared interpretation can be addressed. Interoperability is a matter of degree

rather than a single property, and this thesis is concerned specifically with its semantic level: the point at which exchanged information carries an explicit, machine-interpretable account of what it means. Ontologies supply the formal vocabulary needed to make such meaning explicit, and knowledge graphs populate that vocabulary with the individuals and relations describing an actual domain of interest.

RDF, OWL 2, SPARQL, and SHACL were introduced as the specific technologies used throughout this thesis to represent, reason over, query, and validate such a graph, each necessary and none sufficient on its own. The distinction between foundational, reference, domain, and application ontologies provides the vocabulary needed to describe how independently developed ontologies, built for different disciplines and at different levels of generality, can be related to one another. Competency questions provide the means of judging whether an ontology answers what its users need answered, and are the basis on which the artifacts developed in this thesis are evaluated. Chapter 4 turns to the contributions themselves.

Chapter 4

Synthesis and Contributions

"A composer is a guy who goes around forcing his will on unsuspecting air molecules, often with the assistance of unsuspecting musicians."

Frank Zappa, *The Real Frank Zappa Book*

The two preceding chapters supplied the two halves of the machinery behind Section 1.1: how industrial systems are technically composed, from individual CPSs through SOA to the Arrowhead framework, and how meaning can be made explicit and machine-interpretable through ontologies, knowledge graphs, and the Semantic Web technologies built on them. Neither chapter has yet said anything about what this thesis itself contributes. That is the purpose of this chapter.

Chapter 1 introduced three research questions and mapped the appended papers onto them. Rather than repeating that mapping, this chapter synthesizes what the studies collectively establish, organized around the development of the research program as a whole and, in turn, around each of the three research questions. Some of what this chapter develops belongs to the contribution rather than to shared background knowledge, most notably the IDO and the bridge-ontology mechanism through which it is used. Section 1.1 showed both at work on a single asset without dwelling on why they take the form they do; this chapter supplies that account, at the point where it is needed to understand what the appended papers demonstrate. Publication details, artifacts, and the doctoral candidate's individual contribution to each co-authored paper are recorded separately, in Chapter 6.

4.1 Development of the Research Program

As anticipated in Section 1.3, this section traces the two halves behind the research program in full. Recognizing their shape matters for how the rest of this chapter reads: Papers D–F build on both strands together, not on a chain of increasingly specific prior work.

The first strand, represented by Paper A, is systems-engineering rather than semantic. Paper A investigates brownfield co-engineering through *indistinguishable twins*: a physical and a digital twin integrated via the Arrowhead framework so that external systems cannot distinguish which one they are interacting with. The paper contains no ontology, no RDF, and no reasoning. Its contribution to this thesis is the systems-engineering case that a useful digital representation must participate in the operational environment as a service among others, rather than remain an isolated engineering model consulted separately from the running system. This motivates the run-time, rather than design-time, orientation that Papers D and F later adopt, without itself supplying any of the semantic machinery they use.

The second strand, represented by Papers B and C, develops the semantic-technologies toolkit this thesis relies on, independently of the Arrowhead framework and independently of run-time deployment. Paper B applies OWL 2 reasoning, SPARQL, and, for the first time in this research program, the IDO as a reference ontology, to a hand-authored, design-time description of a pump station. This is where IDO enters the thesis, three papers before the alignment architecture in Paper E makes systematic use of it. Paper C extends this strand from representation to validation, combining OWL 2 reasoning with SHACL constraints in a supply-chain setting and, in doing so, establishing the distinction between what a knowledge graph can infer under an open-world assumption and what must instead be checked as explicitly present under a closed-world reading. This distinction, developed here in a setting unrelated to the Arrowhead framework or industrial run-time systems, becomes the methodological basis on which Paper D later validates run-time snapshots.

Papers D–F draw on both strands at once, which is what makes them the integrated core of the thesis rather than a further, isolated extension. Paper D takes up the run-time orientation established by Paper A and combines it with the reasoning and validation techniques matured in Papers B and C, applying both to a representation, the AFO, generated directly from a live Arrowhead framework deployment rather than hand-authored. Paper E then extends the IDO usage introduced in Paper B from a single, hand-authored model to a formal, reusable alignment architecture that connects AFO to independently maintained plant-design and lifecycle representations. Paper F extends Paper D's run-time representation in a different direction, deriving a second, structurally distinct formal description from the same deployment and, through the *assessor* system it introduces, extending the reasoning-and-validation toolkit from Papers B and C into a further form of operational analysis. The sections that follow take up each research question in turn, tracing in more detail how these contributions relate to it.

4.2 Run-time Semantic Representation of Industrial Systems of Systems

RQ1 asks how a practical, reusable semantic description of an industrial SoS can be generated from the running system and kept current across its lifecycle. This section

addresses that description and what an open, standards-grounded ontology contributes beyond what SOA and digital-twin approaches provide on their own. RQ1 is addressed mainly through Paper D, drawing on the run-time-oriented case established in Paper A, the reasoning and validation techniques developed in Papers B and C, and the extensions in Paper F.

4.2.1 The Arrowhead Framework Ontology

As Chapter 2 and Chapter 3 established, technical and syntactic interoperability, the levels at which the Arrowhead framework's service discovery and orchestration operate, do not by themselves make the meaning of a deployment explicit. A system can locate and bind to a service without anything in that interaction recording why the two systems are related, what would happen if one failed, or how the current configuration compares to one observed earlier. Paper D closes this gap with the AFO, an OWL 2 ontology that formalizes the local clouds, systems, and services of an Arrowhead framework deployment, together with axioms constraining how these may legitimately relate.

Paper D differs in the source of this representation. Rather than being authored by hand at design time, as the plant-design and lifecycle descriptions discussed in Sections 2.1 and 2.4 typically are, AFO is populated directly from a running local cloud, so that the resulting graph describes what is currently configured rather than what was designed or specified. Because this generation is a matter of re-running a pipeline rather than someone remembering to update a separate document, the representation can stay current as the deployment changes, at the cost of describing only what the deployment currently exposes rather than its intent or rationale. Staying current is not the only consideration. Part of what such a description must record, such as which provider a consumer is actually bound to, is settled at run-time and is not written down anywhere at the moment it is decided, so a design-time description could not contain it however diligently it were maintained.

Paper D demonstrates that OWL 2 reasoning and SHACL validation both apply meaningfully to this generated, rather than authored, graph: reasoning enriches a snapshot with facts not explicitly recorded in it, and validation catches malformed or incomplete configurations before they are relied upon. This puts into practice, on a run-time artifact, the same open-world/closed-world distinction Paper C first established in an unrelated supply-chain setting. What this combination enables in practice for diagnosis and traceability is developed further in Section 4.4.

The scope of this contribution is deliberately bounded: AFO covers deployment topology, service dependencies, and core-system roles, not control logic, telemetry, or authorization policy, consistent with the scope set out in Chapter 1. It is a semantic layer over a deployment's configured structure, not a substitute for the framework's operational and security mechanisms.

Paper F extends the contribution in a different direction, deriving a second, structurally distinct formal representation, a SysML v2 model, from the same kind of running deployment, and comparing it directly against the RDF/AFO graph because both trace back to the same operational state. That comparison, and what each representation

proves suited to, is taken up in Section 4.4.5.

Section 2.8 deferred the question of why this thesis does not adopt the AAS as its representational approach, on the grounds that the answer depends on how the AAS structures meaning. An AAS describes a single asset as a container of submodels, each holding elements whose interpretation is fixed by reference to an entry in an external dictionary. Meaning is therefore established by pointing at a published definition rather than by axioms stated in a logic, and the structure is a hierarchy of elements belonging to one asset rather than a graph of relations among many. What has to be made explicit is not primarily what an asset is, but how independently described things stand in relation to one another across disciplines, and those relations have to be derivable rather than enumerated. This is a difference in purpose rather than in quality, and the two are not incompatible. Paper F describes a further Arrowhead framework system that consumes the run-time graph developed here, once extended with product data, and emits AAS representations.

4.3 Reference-Mediated Alignment Across Lifecycle Views

RQ2 asks how a shared reference ontology can mediate alignment between independently developed domain models while preserving each one's own semantics. Where RQ1 concerns a single view of a deployment, its run-time configuration, RQ2 concerns relating that view to others: the plant-design and lifecycle descriptions introduced in Section 2.1 as artifacts maintained by different disciplines, on different schedules, using different vocabularies. Paper E addresses this question primarily, building on the IDO usage first introduced in a single hand-authored model by Paper B.

4.3.1 The Industrial Data Ontology as a Shared Reference Layer

Section 3.4 distinguished foundational, reference, domain, and application ontologies by their generality and intended reuse. The IDO, maintained by the POSC Caesar Association, occupies the reference layer in that typology. It supplies a formal, industrially grounded vocabulary for concepts that recur across disciplines and lifecycle stages, such as systems, functional objects, physical artifacts, and the relations between them, at a level of abstraction general enough to relate independently developed domain models without requiring any of them to adopt a common, detailed schema.

Using a reference ontology this way is a specific architectural choice, and not the only one available. Two independently developed domain models could instead be related through a bespoke, pairwise mapping built specifically for that pair, stating correspondences between their terms directly and without any intermediary.

The decisive consideration between the two is not what they make possible, since both leave the source models intact and both can support cross-domain interpretation. It is what they require as the models change. Under pairwise mapping, every model

revision becomes every other model's concern: a change to how the lifecycle record represents an installation must be reflected in its mapping to the plant-design model and in its mapping to the run-time representation, and each revision requires someone who understands both sides. Under reference mediation, the same change affects only that model's own bridge. The cost shifts rather than disappears, since a revision to the reference ontology may need to be propagated to every bridge at once. What makes the trade favorable is that the reference layer is the most stable element in the arrangement: IDO is a published standard, revised deliberately and on a timescale unrelated to the operational and engineering changes that drive revision in the models it relates. The arrangement concentrates volatile dependencies where change is frequent and local, and shared dependencies where change is rare. This argument holds for IDO itself but not for everything the bridges depend on. IDO does not provide a notion of functional location at the granularity Section 2.2 requires, so the alignment adds a small extension, `lis-ext`, introducing a functional-location class, a class for the physical unit that may occupy it, and a relation between the two. Every hop of the cross-domain trace passes through these three terms. The extension is authored within this work rather than published as part of the standard, so it does not inherit IDO's stability, and a revision to it may need to be propagated to all three bridges just as a revision to IDO would.

A second, more commonly cited consideration is counting. Relating n domain models pairwise requires a mapping for each pair, whereas mediating through a reference layer requires one per model, and a new domain model can be added by relating it once to the reference layer rather than to every model already present. The difference is asymptotic, and it should not be overstated at the scale demonstrated in this thesis: with three domain models the two approaches require the same number of mappings, and the counting argument only begins to favor mediation from the fourth model onward. The maintenance argument above, by contrast, applies from the second model onward.

The reference ontology plays no role beyond this. It does not replace any domain model's own vocabulary, and a domain model aligned to it remains fully queryable in its own terms.

4.3.2 Bridge Ontologies

The mechanism through which a domain model is related to IDO in this thesis is a *bridge ontology*: a separate ontology that imports the domain vocabulary and the reference vocabulary and adds only the axioms needed to relate them, typically stating that a domain-specific class or property is a subclass or subproperty of a more general IDO class or property. One bridge goes slightly further, declaring the property that carries the correspondence between a deployed run-time asset and its functional location, since neither AFO nor IDO provides one. A bridge is deliberately conservative rather than exhaustive. It does not assert that a domain concept and an IDO concept are equivalent, since a domain model routinely carries structure and distinctions that IDO does not represent and has no need to. It instead asserts the narrower claim that the domain concept specializes the reference one, which is sufficient to make instances of the domain

concept discoverable through the reference vocabulary without discarding anything the domain model already expresses.

A second reason for that conservatism is independent of what the domain models happen to represent. The correspondences a bridge states must be found before they can be stated, and identifying them automatically is an established research area [19] that has advanced considerably, most recently through embedding-based [53] and language-model-based [57] matching systems that perform competitively on benchmark alignment tasks. Those benchmarks are dominated by large, lexically rich vocabularies of a kind quite unlike the alignments considered here, where the number of correspondences is small, the terms are technical rather than descriptive, and the engineers who can say what a given term is intended to mean are available to be asked. The more consequential difference is what happens when a correspondence is wrong. A reasoner applies the axioms it is given without judgment, so an incorrect subsumption does not produce an error or an implausible result. It produces a confident one, materialized alongside every other inferred fact and indistinguishable from them, and propagated into every query that crosses it. This places the burden on the correctness of a comparatively small number of statements rather than on the volume of data they govern, and it is why the correspondences used in this thesis are curated and reviewed rather than discovered, and why they assert the least that suffices rather than the most that could be defended.

Because a bridge only adds axioms, and modifies neither the domain ontology nor the reference ontology it connects, the source models remain independently maintained and independently queryable exactly as before the bridge existed. This is what distinguishes alignment from merging as an architectural choice: a merged model requires every contributing source to conform to one shared schema, whereas an aligned model leaves each source's own schema untouched and adds interpretability on top of it. The cost of this choice is that an aligned graph carries weaker consistency guarantees across domains than a single, jointly authored model would, since the domain models were never checked against one another directly, only against the reference layer each independently relates to.

The bridges also determine what the reasoner must support. Most of their axioms are subclass and subproperty statements, which any of the rule sets introduced in Section 3.3.2. The relation between a physical unit and the functional location it occupies is different, since it is derived through a property chain over the lifecycle record. A rule set without property chains, such as OWL-Horst, leaves that segment of the trace absent without reporting an error, so the alignment requires OWL 2 RL or another rule set that supports them. Paper E measures the resulting cost on synthetically generated pump stations. Materialization time grows linearly with the number of stations, at roughly a quarter of a second per station on a single-board computer, and the fixed cost of loading the vocabularies is dominated by the domain ontologies rather than by the 66 axioms of the three bridges.

4.3.3 Cross-Domain Traceability

Taken together, the reference layer and the bridge mechanism answer RQ2 directly. Independently developed domain models can be related without being rewritten or merged, and without any of them adopting another's vocabulary, so that preserving each one's own semantics and achieving cross-domain interpretation are not in tension. At the level of instances, the run-time and lifecycle records do depend on the plant-design individuals they reference, a cost the remainder of this section makes precise. Paper E demonstrates this by relating three independently maintained models to IDO through three bridge ontologies: AFO, representing the run-time deployment, a plant-design model expressed in DEXPI, and the STEP-derived lifecycle model AP4K, covering the plant-design and lifecycle views introduced in Section 2.2 and Section 2.3. No bridge relates any of the three to one another directly. The result is nonetheless sufficient to support traceability across all of them, so that a run-time observation can be followed through to the corresponding engineering and lifecycle information, and back again.

This is achieved at the cost of two kinds of explicit correspondence, and that cost is worth stating precisely, because it is not an artifact of the present implementation. A bridge axiom relates terms rather than individuals. It can establish that a class in one vocabulary is a kind of class in another, and thereby that anything asserted to be the former is also the latter. It cannot establish that a particular individual described in one model stands in some relation to a particular individual described in another. Where two graphs share no identifier, no value, and no assertion, no set of terminological axioms can connect them, because there is nothing for those axioms to act upon. The run-time and plant-design graphs are in exactly this position. The deployment was not configured by consulting the plant design, and the plant design does not record what was deployed, so the relation between a deployed asset and its functional location has to be asserted. The lifecycle view is connected only because its authors reference plant-design individuals directly when recording each installation, which is the statement about a particular deployment at a particular time that Section 2.3 anticipated. Both correspondences therefore point at plant-design individuals. At the level of instances, the plant-design view is where the other two meet, while the reference ontology supplies the vocabulary in which the resulting trace can be queried. What the alignment contributes is not the elimination of these assertions but their reduction to one per deployed asset and one per recorded installation, in place of an integration effort built for every pair of models.

One further step lies outside the alignment itself. Reasoning derives the relation between a physical unit and its functional location for every recorded installation, current and historical alike, since nothing in the shared vocabulary distinguishes the two. The currently installed unit is instead identified by a separate SPARQL update that selects the installation with no recorded end. Its result is derived deployment state rather than entailment, and it holds only for the graph on which the update last ran.

4.4 Capabilities Enabled by the Semantic Layer

RQ3 asks what reasoning, validation, and representation capabilities a run-time-generated, ontology-based knowledge representation enables. Where RQ1 and RQ2 concern building and connecting such a representation, RQ3 asks what it is good for once it exists: a representation that only restates what the underlying systems already expose would add little. Four capabilities recur across the papers that make up this thesis: querying and diagnosis, inference, validation, and traceability.

4.4.1 Querying and Diagnosis

Making a deployment's systems, services, and dependencies explicit in a knowledge graph allows a question to be posed at the level of the composed SoS as a whole, using SPARQL, rather than only at the level of one interface at a time. Paper D shows that this supports diagnostic and impact-oriented questions, such as which systems depend on a given provider or what a failed service would affect, that the Arrowhead framework's own core services do not answer directly, since they are built to support discovery and orchestration rather than retrospective or composed queries over the deployment as a whole.

4.4.2 Inference

OWL 2 reasoning derives facts that follow from an ontology's axioms and a graph's asserted content without those facts being stated explicitly. Paper B first demonstrates this principle in a hand-authored, design-time plant description, and Paper D applies the same principle to a graph generated from a live deployment, showing that inference is not limited to authored, static knowledge but applies equally to a continuously regenerated one. This matters specifically for a run-time representation: relationships that were never explicitly configured, and could not have been, since they only exist as a consequence of how independently developed systems happen to be composed at a given moment, can still be recovered by reasoning over what is currently asserted.

4.4.3 Validation

Inference on its own does not guarantee that a graph is complete or well formed; under an open-world assumption, a missing fact is treated as unknown rather than as evidence that something is wrong. Paper C establishes the resulting distinction between what may be inferred and what must instead be checked as explicitly present, in a supply-chain setting, and Paper D carries the same distinction into the run-time setting, using SHACL to validate generated snapshots before they are relied upon. Across both settings, validation serves a purpose reasoning cannot: catching a graph that is technically consistent but nonetheless incomplete or unsuitable for the task at hand.

4.4.4 Traceability and FMEA-Oriented Analysis

The alignment architecture developed under RQ2 makes a further capability possible: following a run-time observation through to the engineering and lifecycle information needed to interpret it, and back again, discussed in Section 4.3.3. Paper F extends this traceability capability from documentation into operational analysis. A dedicated system consumes the run-time knowledge graph and evaluates a fixed set of queries against it to produce a structural failure-modes-and-effects analysis of the deployment: identifying dependent services, components shared across otherwise separate functions, and structural conditions such as a command issued without a corresponding feedback signal being consumed. This does not automate the engineering judgment a FMEA ultimately requires; severity, occurrence, and detection ratings remain the analyst's responsibility. What is generated, and regenerated as the deployment changes, is the structural basis the analysis depends on, which complements rather than replaces ontology-based work on FMEA knowledge representation.

4.4.5 Representational Form

The capabilities above concern what can be done with a representation of this kind. RQ3 also asks what the choice of representation itself contributes, and answering that requires a comparison rather than a demonstration. Paper F provides one by generating two artifacts from the same running deployment: an RDF knowledge graph conforming to AFO, and a SysML v2 model conforming to a corresponding library. Because both are derived from the same deployment state rather than authored separately, any difference between them is attributable to the representational paradigm rather than to differences in scope, source data, or the intentions of whoever produced them.

Evaluated against a shared set of competency questions, the two representations proved suited to different things. The RDF representation supported semantic integration, querying, and reasoning across system boundaries. The SysML representation was better suited to expressing system structure, interfaces, and behavior. The conclusion drawn is that the two are complementary rather than competing, which bears on how the rest of this thesis should be read: the case for an ontology-based representation made under RQ1 and RQ2 is a case for a specific set of capabilities, not a claim that this formalism is preferable to others for every purpose. It also indicates that generating a formal description from a running deployment is not tied to the ontology-based approach in particular, since the same run-time source proved sufficient to produce a structural model in an entirely different paradigm.

4.5 Chapter Summary

This chapter has synthesized how the appended papers, developed along two initially separate strands, systems-engineering and semantic representation, converge on the three research questions posed in Chapter 1. AFO answers RQ1 by showing that a run-

time-generated, standards-grounded ontology adds a machine-interpretable account of a deployment's configuration beyond what service-level interoperability provides on its own. A shared reference ontology, connected to independently maintained domain models through minimal bridge ontologies, answers RQ2 by making cross-domain traceability possible without requiring any source model to be rewritten or merged. Querying, inference, validation, and traceability, demonstrated individually and in combination across the papers, answer RQ3 by showing what such a representation makes possible once it exists: diagnosis at the level of the composed system, knowledge recovered without being explicitly configured, incompleteness caught before it is relied upon, and operational analysis, including a structural basis for FMEA, generated directly from the running deployment rather than maintained by hand. Publication details, artifacts, and the doctoral candidate's individual contribution to each work are recorded in Chapter 6.

Chapter 5

Reflections and Future Work

"There is no fate but what we make for ourselves."

—John Connor, *The Terminator*

This thesis set out from a simple observation: that industrial systems can be technically connected without thereby agreeing on what the information they exchange means, and that this residual gap, not the absence of connectivity, is where semantic interoperability becomes necessary. Section 1.1 showed what closing that gap looks like in practice, Chapter 1 posed three research questions to investigate it, and Chapter 4 synthesized how the appended papers address them. This chapter first sets out the delimitations within which all of that was done and checks each against what was actually built, then revisits each research question in turn, and closes with directions for future work.

5.1 Scope and Limitations, in Retrospect

This thesis is delimited along five dimensions, introduced in Chapter 1 and assessed here against what the completed work demonstrates. They are brought together here so that each can be read directly against the evidence of the appended papers rather than only as a boundary declared in advance.

- **Lifecycle stages.** The lifecycle stages covered are plant design, asset lifecycle records, and run-time operation. Early-stage concept definition and requirements elicitation are not addressed. Two of the appended papers represent existing requirements and specifications as ontology-based, machine-checkable constraints, but none derives or elicits those requirements from stakeholder needs.
- **Deployment scale.** The demonstrated deployments are single-organization and laboratory- or demonstrator-scale in every case. None of the appended papers evaluates the approach against a real, externally operated industrial deployment.
- **Cross-organizational interoperability.** This is not directly evaluated in this thesis. Two of the appended papers demonstrate alignment mechanisms that do not require a

shared owner or a merged model, one of them explicitly separating data by originating party, which suggests the approach is not architecturally restricted to a single organization. Neither paper, however, validates this across genuinely independent, externally operated organizations, and one states this explicitly as future work.

- **End of life.** End-of-life and decommissioning are discussed conceptually in relation to circular-economy goals but are not demonstrated or evaluated in any of the appended papers.

- **Ground truth.** The semantic representations in this thesis describe the configured or declared state of a system, not independently verified physical ground truth. Whether a deployment behaves as its semantic description claims remains outside the scope of the present work.

Taking each in turn.

The lifecycle-stage delimitation holds as stated. Plant design, lifecycle records, and run-time operation are all represented, through the DEXPI-based, AP4K, and AFO views respectively, but nothing in the appended papers derives or elicits requirements from stakeholder needs; where requirements appear at all, in Paper C, they are represented as constraints to be checked against, not generated from an elicitation process. The delimitation was accurate, and nothing in the completed work pushes against it.

The deployment-scale delimitation also holds, and more uniformly than it might have. Every demonstrator across Papers D, E, and F is single-organization and laboratory- or demonstrator-scale: a climate-control local cloud, a laboratory pump station, and a climate-control deployment extended to a second, somewhat larger operational cottage. That second deployment is a partial exception worth noting precisely because of how limited it is: applying the same generators to eleven systems and fifty services, and federating it with an independently generated pump-station graph, shows that the approach does not immediately break outside a single small demonstrator. Paper E approaches the question from a different direction, evaluating how the cost of reasoning over the aligned representation grows as the volume of instance data increases, using synthetically generated data at increasing scale. Neither of these constitutes the externally operated, independently developed deployment the original delimitation excluded. One enlarges the demonstrator and the other measures computational behavior under load, and neither introduces the organizational and operational realities an industrial deployment would bring with it. None of the appended papers claims otherwise.

Cross-organizational interoperability was delimited as not directly evaluated, with a note that some mechanisms suggested the approach was not architecturally restricted to a single organization. This is confirmed rather than merely repeated by the completed work. Paper C's use of named graphs to separate assertions by originating business partner, producer and supplier alike, demonstrates a mechanism that does not require a shared owner, exactly as anticipated. Paper E's bridge ontologies are similarly agnostic to organizational boundary in their design, since a bridge acts on vocabulary and is indifferent to which organization asserted the data it applies to. Neither paper, however,

was evaluated against genuinely independent, externally operated organizations, and the delimitation's caution on this point stands.

The end-of-life delimitation also holds without qualification: Paper C engages circular-economy goals conceptually, through its DPP-adjacent framing, but no appended paper demonstrates or evaluates decommissioning.

The ground-truth delimitation is the one the completed work speaks to most directly, and most explicitly. This was not left implicit for the reader to infer; both Paper D and Paper F say so themselves. Paper D's own discussion is explicit that its demonstrator is a proof-of-concept evaluation on an author-developed deployment, not external validation across independently developed Arrowhead deployments. Paper F goes further, stating plainly that trust in a run-time-generated representation is inseparable from trust in the pipeline that generates it, and that the artifacts inspected in this thesis were checked for structural completeness rather than full semantic conformance. The delimitation set out in Chapter 1, that these representations describe configured or declared state rather than independently verified physical reality, is therefore not merely an a priori boundary but one the appended papers themselves confirm and reason about directly.

Taken together, the five delimitations held. That is a weaker result than it may appear, since a scope drawn in advance can hold simply because nothing was attempted that would have tested it. The more useful observation is that the laboratory-scale, single-organization, configured-state character of the contribution is more sharply visible now than it could have been when the scope was first drawn, confirmed by the appended papers' own discussions rather than only asserted in advance of them.

One limitation is not among the five, and became apparent only once the alignment was complete. This thesis argues that hand-authored descriptions drift from the systems they describe, and answers that argument for the run-time view by generating it rather than authoring it. The plant-design and lifecycle views it aligns to are not generated. They are maintained by the disciplines that own them and revised when those disciplines revise them, and are therefore subject to precisely the drift the thesis set out to address. The alignment relates three views, of which one stays current by construction and two do not. Nothing in the appended papers is wrong on this point, since each addresses the view it develops, but the asymmetry is a property of the combined architecture rather than of any single paper. It bounds what cross-domain traceability can be relied upon to show, since a trace is only as current as the least current of the models it crosses.

Two further dependencies bound the trace in the same way. The identification of the currently installed unit is produced by an update step rather than by reasoning, so it is only as current as the last time that step ran. The explicit correspondences from the run-time and lifecycle views reference plant-design individuals directly, and they remain valid only as long as those individuals keep their identifiers across design revisions. A correspondence of this kind is what Chapter 1 criticized in hand-maintained conversion tables, and the alignment reduces the number of such statements without making them immune to the same drift.

5.2 Revisiting the Research Questions

RQ1 asked how a practical, reusable semantic description of an industrial SoS can be generated from the running system and kept current across its lifecycle. Section 4.2 answered this through AFO: a run-time generated, rather than hand-authored, ontology adds a machine-interpretable account of a deployment's configuration, one that can be queried, validated, and reasoned over independently of the interfaces through which the deployment itself operates. This is not something SOA or a design-time digital twin provides on its own. Service discovery and orchestration make a deployment operable, not explicable, and a digital twin synchronized with a running system is not thereby related to the engineering and lifecycle knowledge surrounding it. The answer to RQ1 is consequently that such a contribution is achievable, and that its value lies specifically in being generated from, and kept current with, the running system rather than authored once and left to drift.

A single system's run-time configuration is, however, only one of the views a production system accumulates over its lifecycle, and RQ2 turned to the harder problem this raises: how a shared reference ontology can mediate alignment between independently developed domain models while preserving each model's own semantics. Section 4.3 answered this through a reference layer and a small set of additive bridge ontologies: independently maintained domain models can be related without being rewritten or merged, and without any of them adopting another's vocabulary, at the cost of one explicit correspondence per deployed asset and one per recorded installation, rather than a bespoke integration built for every pair of models involved. Preserving domain-specific semantics and achieving cross-domain traceability, which might appear to be competing goals, turn out not to be in tension once alignment is treated as additive rather than as replacement.

Establishing that a run-time representation can be built, and that it can be related to other views without losing its own semantics, still leaves open what such a representation is actually good for once it exists, which is the question RQ3 posed: what reasoning, validation, and representation capabilities a run-time-generated, ontology-based knowledge representation enables. Section 4.4 identified four such capabilities across the appended papers. Querying and diagnosis operate at the level of a composed system rather than a single interface. Inference recovers relationships that were never explicitly configured. Validation catches an incomplete or malformed graph before it is relied upon. And traceability follows a run-time observation through to the engineering and lifecycle information needed to interpret it, extended in one paper into a regenerable structural basis for FMEA-oriented analysis. In the demonstrated deployments, these capabilities do not arise from the underlying systems in isolation; each depends on the semantic layer this thesis develops, which is what answers RQ3. RQ3 also asks what the representation itself contributes, and Section 4.4.5 addressed this by comparing the ontology-based representation against a structural model generated from the same deployment. The two proved suited to different kinds of analysis, which qualifies the answer to RQ3 rather than extending it: the capabilities above follow from this particular formalism, and are not a claim that it is preferable to others for every purpose.

5.3 Future Work

Read together rather than paper by paper, the future work proposed across Papers D, E, and F clusters around a small number of recurring concerns, and it is more informative to trace those threads than to repeat each paper's own list.

The most immediate thread is scale and external validation. Every demonstrator in this thesis is laboratory- or demonstrator-scale and single-organization, a limitation Section 5.1 already discussed in retrospect, and each of the relevant papers proposes a version of the same next step: evaluating the approach against richer plant structures, multiple operational clouds, longer maintenance histories, and deployments the candidate did not build. This is not three separate concerns but one, viewed from the run-time, alignment, and dual-representation angles respectively, and addressing it in any one of the three would likely inform the other two.

A second thread concerns the points in this thesis that still depend on a person rather than a reasoner. The alignment architecture developed for RQ2 requires one explicit correspondence per deployed asset and one per recorded installation. The second is recorded as part of ordinary lifecycle practice, but the first is an additional burden on the integrator, and Paper E suggests it could in principle be derived from commissioning records or tag conventions instead. A related, though distinct, concern is raised in Paper D's discussion of validation strategy: deciding when a SHACL violation should reject a snapshot outright versus when a historically imperfect one should be preserved for root-cause analysis is currently a matter of applying SHACL selectively, a judgment call rather than an automated one. Reducing either dependency would not remove the need for human oversight of the semantic layer, but it would shift where that oversight is spent.

A third thread follows directly from the ground-truth limitation discussed in Section 5.1. The representations developed in this thesis describe configured or declared state, and Paper E's own future work names the natural next step explicitly: combining the semantic backbone with time-series observations from the running plant, moving from configured traceability toward behavioral validation. This would not replace the structural, configuration-level knowledge this thesis develops, but would test it against what a deployment actually does rather than only what it declares itself to be configured as, directly addressing the gap between declared and physical state that Section 5.1 identified as the thesis's most explicit self-acknowledged limitation.

A fourth thread concerns the trustworthiness of the generation pipeline itself, raised most directly in Paper F's discussion but implicit wherever a representation is generated rather than authored. Because the validity of a run-time-derived graph or model depends on the fidelity of the process that produces it, strengthening that pipeline, through conformance testing, assurance mechanisms, and, where operational knowledge graphs may reveal sensitive information, appropriate security and disclosure controls, is future work in its own right, separate from extending what the representations themselves cover. This thread is a reminder that the semantic layer this thesis develops is only as trustworthy as the software that generates it, a dependency this thesis has made explicit rather than

hidden.

A final thread concerns reach beyond the standards this thesis has aligned. Chapter 1 deliberately did not adopt the AAS as this thesis's semantic layer, treating the run-time and lifecycle representations developed here as an alternative to it rather than a replacement. Paper E's own future work proposes extending the run-time bridge to cover AAS submodels alongside AFO, since both target the same run-time layer of the co-engineering problem from different standards communities. Pursued to completion, this would not revise the position taken in Chapter 1 so much as test it, by establishing whether the alignment architecture developed for DEXPI and STEP extends as cleanly to a standard this thesis chose not to build on directly.

Together, these five threads, scale, manual dependency, behavioral validation, pipeline trust, and standards reach, do not point toward a single next paper so much as toward the kind of program this thesis itself grew out of: incremental, evaluated through concrete demonstrators, and answerable to the same standard of evidence this thesis has tried to hold itself to throughout.

Chapter 6

Appended Papers and Research Artifacts

6.1 Papers, Research Artifacts, and Individual Contributions

Chapter 4 synthesized the collective contribution of the thesis. This chapter documents the individual research outputs on which that synthesis is based: their publication status, scope, associated research artifacts, and the doctoral candidate's contribution to each work.

6.1.1 Appended Papers

Paper A

Title	Transforming Brownfield Factories: Unleashing the Potential With Co-Engineering and Virtual Commissioning
Authors	Oskar Wintercorn, Khalid Atta, Peter Jeppsson, Cristina Paniagua, and Jan van Deventer
Published in	IECON 2023 – 49th Annual Conference of the IEEE Industrial Electronics Society
Summary	This paper proposes indistinguishable twins: a digital twin and a physical twin that are integrated such that external systems cannot distinguish whether they are interacting with the real system or its simulated counterpart. The twins are made interoperable through a local cloud approach using the Eclipse Arrowhead framework, enabling service-based interaction between the twins and with external legacy

systems. A proof-of-concept is demonstrated using a fischertechnik model factory controlled by a Siemens PLC, where a Siemens NX/MCD simulation is connected to the PLC logic to emulate physical behavior.

Motivation Brownfield factories must integrate replacements and upgrades into legacy production lines under time pressure, but co-engineering and virtual commissioning practices are significantly more mature for greenfield contexts. The paper targets the gap where digital twins exist but do not provide robust interoperability with legacy systems and diverse industrial protocols, which makes it difficult to evaluate and develop upgrades before the physical installation is available.

Contribution I designed the digital twin and programmed it using the Golang implementation of the Arrowhead framework. Khalid Atta assisted with the PLC programming and setup. Peter Jeppsson assisted with the design and simulation of the digital twin. Cristina Paniagua and Jan van Deventer assisted with comments and discussion during the writing process.

Paper B

Title Unlocking the Power of Digital Transformation: The Role of Ontologies

Authors Oskar Wintercorn, Jan van Deventer, and Cristina Paniagua

Published in 2025 IEEE International Systems Conference (SysCon)

Summary This paper argues that industrial digital transformation is limited less by data availability and more by the lack of shared meaning across heterogeneous sources. It demonstrates how ontologies and knowledge graphs can improve stakeholder communication and semantic interoperability by providing explicit, machine-interpretable representations. Using a pump-station case study, it illustrates how axioms, reasoning, and SPARQL queries enable the plant description to remain useful across lifecycle phases and to support consistent interpretation across systems and workflows.

Motivation Industrial plants increasingly rely on diverse data sources, formats, and standards, which leads to abundant information but insufficient context for trustworthy reuse and collaboration. The paper motivates ontologies as a way to move from information to knowledge by enabling machine-level interpretation, inference, and inconsistency detection.

Contribution I modeled the knowledge graph using IDO as the reference ontology, termed an upper ontology in that paper, and wrote the paper. Jan

	van Deventer and Cristina Paniagua were part of the writing process, with discussion and guidance.

Paper C

Title	From Specification to Compliance: A Semantic Approach to Validating Industrial Supply Chains
Authors	Oskar Wintercorn and Jan van Deventer
Published in	7th International Conference on Industry of the Future and Smart Manufacturing, 2026
Summary	This paper presents a semantic framework for validating product and supply information in industrial value streams by combining ontology-based reasoning with SHACL-based constraint validation. Requirements and supplied product descriptions are represented in a knowledge graph grounded in reference ontologies, enabling inference of requirement satisfaction and closed-world validation of completeness and integrity. A demonstrative use case shows how the approach supports traceability, improves data quality, and strengthens compliance by replacing blind trust in supplier specifications with machine-checkable validation.
Motivation	Supply-chain and supplier-provided specifications are often heterogeneous, difficult to validate, and may be incomplete or inconsistent, which creates quality risks and compliance burdens. The work is additionally motivated by emerging initiatives such as the Digital Product Passport, which increase the demand for traceable, verifiable, and interoperable product information across organizations.
Contribution	I designed and created the approach combining IDO and ChEBI with a knowledge graph of a small experimental product. Jan van Deventer assisted with input and as a discussant throughout the writing process.

Paper D

Title	The Arrowhead Framework Ontology
Authors	Oskar Wintercorn and Jan van Deventer
Published in	IEEE Open Journal of the Industrial Electronics Society, vol. 7, 2026.
Summary	This paper introduces the AFO as an OWL 2 Semantic Web model that makes Arrowhead framework local clouds explicit and queryable as operational knowledge graphs. Using a climate-control SoS demonstrator, the paper exports timestamped RDF snapshots from live

systems, ingests them into GraphDB, validates them with SHACL, and applies OWL 2 reasoning to derive additional facts about structure and service dependencies. The resulting representation supports SPARQL-based diagnosis and impact analysis, digital threads over time, and experimentation with a natural-language interface that generates SPARQL queries.

Motivation Modern industrial applications are composed dynamically from independently developed systems that discover each other and bind at run-time. Diagnosing and understanding the behavior of such composed SoS requires contextual knowledge that is rarely explicit in any single constituent system. The paper targets this gap by introducing an explicit semantic layer that can be added non-intrusively on top of existing Arrowhead framework deployments.

Contribution I designed and implemented the AFO, including the conceptual model and OWL 2 axiomatization. I implemented the pipeline for exporting and ingesting run-time snapshots and for performing SHACL validation and OWL 2 reasoning in the knowledge graph environment. Jan van Deventer developed the mbaigo Arrowhead framework implementation used as a basis for this work and contributed through discussion, feedback, and guidance during the writing process.

Paper E

Title Ontology Alignment for Co-Engineering: Integrating Run-time, Plant Design, and Lifecycle Views Through the Industrial Data Ontology

Authors Oskar Wintercorn and Jan van Deventer

Published in Submitted to IEEE Open Journal of the Industrial Electronics Society

Summary This paper presents an IDO-mediated alignment architecture that connects three independently developed domain ontologies: the AFO, a DEXPI-based plant-design representation, and a STEP-derived lifecycle ontology (AP4K), without modifying any source model. Three OWL 2 bridge ontologies align each domain view to the IDO, enabling cross-domain SPARQL queries that traverse the run-time, plant-design, and lifecycle views. A laboratory pump-station demonstrator shows end-to-end traceability from a registered run-time service to its P&ID equipment tag and the serial number of the currently installed physical unit.

Motivation As industrial automation shifts toward service-oriented, cross-layer deployments, interoperability problems increasingly arise from mismatched assumptions about what data and services mean across tools,

vendors, and lifecycle phases. Connecting every pair of domain models independently does not scale and produces brittle mappings that break as deployments evolve. The paper motivates alignment through a shared industrial reference ontology as a reusable mechanism that preserves domain-specific semantics while enabling cross-domain interpretation.

Contribution I designed the alignment architecture and implemented the three OWL 2 bridge ontologies connecting AFO, the DEXPI-based plant-design model, and AP4K to IDO, together with the `lis-ext` extension. I developed the demonstrator knowledge graphs, the AP4K-based lifecycle instance model, the synthetic instance generator, and the cross-domain SPARQL queries used to evaluate the alignment. The OWL representation of DEXPI used in the paper was developed by Torbjörn Holm / Plant&Bytes / Semantum, and STEP AP4K by Torbjörn Holm and Alexander Svensson. Jan van Deventer contributed through discussion, feedback, and guidance during the writing process.

Paper F

Title One System of Systems, Two Paradigms: Runtime-Generated RDF and SysML v2 Representations of an IIoT Deployment

Authors Oskar Wintercorn, Franziska Sievert, and Jan van Deventer

Published in To be submitted

Summary This paper generates two formal representations, an RDF graph conforming to the AFO and a SysML v2 model conforming to a minimal Arrowhead framework modeling library, directly from the same running Arrowhead framework deployment. Because both artifacts originate from the same deployment state, differences between them can be attributed to the underlying modeling paradigm rather than to differences in scope or authoring decisions. The two representations are compared through a shared set of competency questions, showing that each answers a distinct class of questions the other cannot, and that the two are complementary rather than redundant.

Motivation Formal descriptions of industrial deployments are typically authored before integration and frozen at handover, after which they drift from the system they were meant to describe. The paper addresses this by generating both representations from the operational deployment itself, and by asking which questions each representation paradigm, open-world and graph-based versus closed-world and structural, is actually suited to answering.

Contribution I designed the AFO in previous works and performed the RDF portion of this work. I co-authored the paper with Franziska Sievert. Franziska Sievert performed the SysML v2 portion of this work and co-wrote the paper. Jan van Deventer contributed through discussion, feedback, and guidance during the writing process.

6.1.2 Research Artifacts

Artifact A

Title Pump Station Use Case

Authors Oskar Wintercorn

Available at `https://github.com/PCA-POSC-Caesar-Association/IDO_Use_Case_Pumpstation`

Summary A use case modeling a pump station according to IDO.

Motivation To provide a concrete, minimal, and reusable example of how IDO can be applied to model an industrial pump station in a way that supports semantic interoperability and query-based reuse across lifecycle phases.

Contribution I created the use case model, including the IDO-based conceptualization, the instance data, and example queries demonstrating how the model can be used as a baseline for later demonstrators and papers.

Artifact B

Title Digital Product Description

Authors Oskar Wintercorn

Available at `https://github.com/PCA-POSC-Caesar-Association/IDO_Use_Case_Supply_Chain_in_Chemical_Industry`

Summary A chemical product modeled as a digital thread according to ontologies such as IDO and ChEBI.

Motivation To explore how a chemical product can be represented as an interoperable digital thread, where identifiers, composition, and properties are explicitly modeled and can be validated and traced across organizational boundaries.

Contribution I designed and implemented the knowledge graph for the product description, integrating IDO with ChEBI to represent the product and its chemical composition, and provided example queries and validation patterns that informed later work on compliance-oriented validation.

Artifact C

Title	The Arrowhead Framework Ontology
Authors	Oskar Wintercorn
Available at	`https://github.com/oskwin/Arrowhead-Framework-Ontology`
Summary	The AFO source ontology.
Motivation	To provide a reusable and machine-interpretable representation of an Arrowhead framework local cloud that makes systems, services, and their relationships explicit, enabling query-based diagnosis, impact analysis, and traceability without modifying the deployed systems. The artifact serves as the semantic backbone for the later demonstrators and for aligning run-time service ecosystems to engineering-time models.
Contribution	I created and maintained the ontology and implemented supporting tooling on top of the mbaigo implementation of the Arrowhead framework, developed by Jan van Deventer.

Artifact D

Title	The Arrowhead Framework Ontology Alignment
Authors	Oskar Wintercorn
Available at	`https://github.com/oskwin/Arrowhead-Framework-Ontology`
Summary	An alignment extension for the AFO to IDO.
Motivation	To bridge engineering-time plant descriptions and run-time service ecosystems by connecting the AFO to a shared reference ontology (IDO). The alignment enables cross-domain querying and inference through IDO-level abstractions and avoids brittle pairwise mappings, making the approach more scalable as models and deployments evolve.
Contribution	I created and maintained the ontology and implemented supporting tooling using the mbaigo Arrowhead framework, developed by Jan van Deventer.

Artifact E

Title	Ontology Alignment for Co-Engineering: Pump-Station Use Case
Authors	Oskar Wintercorn
Available at	`https://github.com/oskwin/afo-alignment-use-case`

Summary The three bridge ontologies aligning the AFO, DEXPI, and AP4K to IDO, together with the `lis-ext` extension, the pump-station instance data, the synthetic instance generator, and the SPARQL queries used in Paper E. The OWL representations of DEXPI and AP4K that two of the bridges target are not included, since they are not publicly available at the time of writing.

Motivation To make the alignment evaluated in Paper E inspectable and reusable, including the bridge axioms, the explicit correspondences they act on, and the queries through which the cross-domain trace is demonstrated. The synthetic instance generator allows the scalability experiment to be repeated at different scales once the target ontologies are available.

Contribution I created the bridge ontologies, the `lis-ext` extension, the instance data, the synthetic instance generator, and the queries. The OWL representation of DEXPI was developed by Torbjörn Holm / Plant&Bytes / Semantum, and STEP AP4K by Torbjörn Holm and Alexander Svensson.

6.1.3 Further Impact

This section presents the remainder of publications that have been impacted by this research. The main contributions of these papers are presented by their first authors and is not for this research to claim.

Paper G

Title Visualization, Representation, and Interoperation of Digital Twins with Open-Source Software Frameworks

Authors Markus Kelanti, Henri Bomström, Sarthak Acharaya, Oskar Wintercorn, Jan van Deventer, and Tero Päivärinta

Published in Automaatiopäivät 2023 – Automation Days 2023: Book of Extended Abstracts

Summary This paper investigates the automatic propagation of changes between the digital and physical counterparts of a miniature model factory. The aim is a simulation that mimics the physical factory closely enough that changes made in one are reflected in the other. The work builds on open-source frameworks and platforms, including the Arrowhead framework, Vorto, Ditto, and Unreal Engine [33].

Contribution A digital twin environment is combined with other open-source technologies so that maintenance workers can interact with a physical

asset through its digital counterpart. I developed the digital twin and its communication with the Arrowhead framework.

Paper H

Title Twins Interoperability through Service Oriented Architecture: A use-case of Industry 4.0

Authors Sarthak Acharaya, Oskar Wintercorn, Aparajita Tripathy, Muhammad Hanif, Jan van Deventer, and Tero Päivärinta

Published in Proceedings of the Annual Symposium of Computer Science 2023, CEUR Workshop Proceedings, vol. 3506

Summary This work extends the scenario in Paper A by adding a document-based digital twin. The result is a resilient twin that falls back on available documentation where live data is unavailable.

Contribution Building on the digital twin of a fischertechnik factory created in Paper A, this work explores the benefits of a document-based digital twin. The aim is to support predictive maintenance by making factory monitoring more reliable. As in Paper G, I created the digital twin and its communication with the Arrowhead framework.

References

[1] Iec 81346-1:2022 industrial systems, installations and equipment and industrial products — structuring principles and reference designations — part 1: Basic rules, 2022.

[2] Dexpi p&id specification version 1.4, 2024.

[3] Tasnim A. Abdel-Aty and Elisa Negri. Conceptualizing the digital thread for smart manufacturing: a systematic literature review. *Journal of Intelligent Manufacturing*, 35:3629–3653, 2024.

[4] Dean Allemang and James Hendler. *Semantic Web for the Working Ontologist: Effective Modeling in RDFS and OWL*. Morgan Kaufmann, 2nd edition, 2011.

[5] Abdulrahman Alqoud, Dirk Schaefer, and Jelena Milisavljevic-Syed. Industry 4.0: a systematic review of legacy manufacturing system digital retrofitting. *Manufacturing Review*, 9, 2022.

[6] Raphael Barbau, Sylvere Krima, Sudarsan Rachuri, Anantha Narayanan, Xenia Fiorentini, Sebti Foufou, and Ram D. Sriram. Ontostep: Enriching product model data using ontologies. *Computer-Aided Design*, 44(6):575–590, 2012.

[7] Basic Formal Ontology. Basic formal ontology (bfo), 2024. Accessed: 2024-10-21.

[8] Rafael Batres, Matthew West, David Leal, David Price, Masaki Katsube, Yukiyasu Shimada, Tetsuo Fuchino, and Yuji Naka. An upper ontology based on iso 15926. *Computers & Chemical Engineering*, 31(5-6):519–534, 2007.

[9] Gordon Blair, Nelly Bencomo, and Robert B. France. Models@ run.time. *Computer*, 42(10):22–27, 2009.

[10] J. Boardman and B. Sauser. System of systems - the meaning of of. In *2006 IEEE/SMC International Conference on System of Systems Engineering*, pages 6 pp.–, 2006.

[11] Hugh Boyes, Bil Hallaq, Joe Cunningham, and Tim Watson. The industrial internet of things (iiot): An analysis framework. *Computers in Industry*, 101:1–12, 2018.

[12] Dan Brickley and R.V. Guha. RDF schema 1.1. W3C Recommendation, 2014.

[13] Flavio Cirillo, Gökhan Solmaz, Indré Berzanskytė, Martin Bauer, Luis Sanchez, and Antonio Skarmeta. A standard-based open source IoT platform: FIWARE. *Computer Networks*, 169:106980, 2020.

[14] Richard Cyganiak, David Wood, and Markus Lanthaler. RDF 1.1 Concepts and Abstract Syntax. W3c recommendation, World Wide Web Consortium (W3C), 2 2014. Accessed: 2026-08-27.

[15] Laura Daniele, Frank den Hartog, and Jasper Roes. Created in close interaction with the industry: the smart appliances reference (saref) ontology. In *Formal Ontologies Meet Industry (FOMI 2015)*, volume 225 of *Lecture Notes in Business Information Processing*, pages 100–112. Springer, 2015.

[16] Jerker Delsing, editor. *IoT Automation: Arrowhead Framework*. CRC Press, 2017.

[17] Milos Drobnjakovic, Boonserm Kulvatunyou, Farhad Ameri, Chris Will, Barry Smith, and Albert T. Jones. The industrial ontologies foundry (iof) core ontology. In *Proceedings of the 12th International Workshop on Formal Ontologies Meet Industry (FOMI 2022)*, volume 3240 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2022.

[18] Thomas Erl. *Service-oriented architecture*, volume 8. Pearson India, 2005.

[19] Jérôme Euzenat and Pavel Shvaiko. *Ontology Matching*. Springer, Berlin, Heidelberg, 2nd edition, 2013.

[20] Sandra Fillinger, Henning Bonart, Wolfgang Welscher, Erik Esche, and Jens-Uwe Repke. Improving interoperability of engineering tools – data exchange in plant design. *Chemie Ingenieur Technik*, 89(11):1454–1463, 2017.

[21] Alvaro Luis Fraga, Marcela Vegetti, and Horacio Pascual Leone. Ontology-based solutions for interoperability among product lifecycle management systems: A systematic literature review. *Journal of Industrial Information Integration*, 20:100176, 2020.

[22] Michael Grieves and John Vickers. Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems. In *Transdisciplinary perspectives on complex systems: New findings and approaches*, pages 85–113. Springer, 2016.

[23] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[24] Michael Gruninger. Methodology for the design and evaluation of ontologies. In *Proc. IJCAI'95, Workshop on Basic Ontological Issues in Knowledge Sharing*, 1995.

[25] Nicola Guarino. Formal ontology and information systems. In *Proceedings of the 1st International Conference on Formal Ontology in Information Systems (FOIS'98)*, pages 3–15, Trento, Italy, 1998. IOS Press.

[26] Steve Harris and Andy Seaborne. SPARQL 1.1 Query Language. W3c recommendation, World Wide Web Consortium (W3C), 3 2013. Accessed: 2026-08-27.

[27] Thomas D. Hedberg, Manas Bajaj, and Jaime A. Camelio. Using graphs to link data across the product lifecycle for enabling smart manufacturing digital threads. *Journal of Computing and Information Science in Engineering*, 20(1):011011, 2020.

[28] International Electrotechnical Commission. IEC 62264-1:2013 enterprise-control system integration – part 1: Models and terminology. Standard, International Electrotechnical Commission, Geneva, Switzerland, 2013.

[29] International Electrotechnical Commission. IEC 63278-1:2023 asset administration shell for industrial applications – part 1: Asset administration shell structure. Technical report, IEC, 2023.

[30] International Organization for Standardization. ISO/DIS 23726-3: Automation systems and integration – ontology-based interoperability – part 3: Industrial data ontology. Technical report, ISO, 2024.

[31] Krzysztof Janowicz, Armin Haller, Simon J. D. Cox, Danh Le Phuoc, and Maxime Lefrançois. SOSA: A lightweight ontology for sensors, observations, samples, and actuators. *Journal of Web Semantics*, 56:1–10, 2019.

[32] Marco Kainzner, Christoph Klösch, Dominik Filipiak, Tek Raj Chhetri, Anna Fensel, and Jorge Martinez-Gil. Towards reusable ontology alignment for manufacturing maintenance. In *SEMANTiCS 2021 Companion Volume*, volume 2941 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2021.

[33] Markus Kelanti, Henri Bomström, Sarthak Acharya, Oskar Wintercorn, Jan van Deventer, and Tero Päivärinta. Visualisation, representation and interoperation of digital twins with open-source software frameworks. *Automaatiopäivät 2023-Automation Days 2023: Book of Extended Abstracts*, 2023.

[34] Holger Knublauch and Dimitris Kontokostas. Shapes Constraint Language (SHACL). W3c recommendation, World Wide Web Consortium (W3C), 7 2017. Accessed: 2026-08-27.

[35] Juha Kortelainen, Tatiana Minav, and Kari Tammi. Digital twin—the dream and the reality. *Frontiers in the Internet of Things*, 2, 2023.

[36] Edward A. Lee. Cyber physical systems: Design challenges. In *2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC)*, pages 363–369, 2008.

[37] Jay Lee, Behrad Bagheri, and Hung-An Kao. A cyber-physical systems architecture for industry 4.0-based manufacturing systems. *Manufacturing Letters*, 3:18–23, 2015.

[38] Wolfgang Mahnke, Stefan-Helmut Leitner, and Matthias Damm. *OPC Unified Architecture*. Springer, 2009.

[39] Mark W Maier. Architecting principles for systems-of-systems. *Systems Engineering: The Journal of the International Council on Systems Engineering*, 1(4):267–284, 1998.

[40] Laetitia V. Monnier, Guodong Shao, and Sebti Foufou. A methodology for digital twins of product lifecycle supported by digital thread. In *Proceedings of the ASME 2022 International Mechanical Engineering Congress and Exposition (IMECE2022)*, number IMECE2022-95182, Columbus, OH, USA, October 2022. ASME.

[41] L. Monostori, B. Kádár, T. Bauernhansl, S. Kondoh, S. Kumara, G. Reinhart, O. Sauer, G. Schuh, W. Sihn, and K. Ueda. Cyber-physical systems in manufacturing. *CIRP Annals*, 65(2):621–641, 2016.

[42] Jan Morbach, Aidong Yang, and Wolfgang Marquardt. Ontocape—a large-scale ontology for chemical process engineering. *Engineering Applications of Artificial Intelligence*, 20(2):147–161, 2007.

[43] Boris Motik, Peter F. Patel-Schneider, and Bernardo Cuenca Grau. OWL 2 web ontology language: Structural specification and functional-style syntax (second edition). W3C recommendation, World Wide Web Consortium, December 2012. 11 December 2012.

[44] Zoltan Nagy, Ryan Wisnesky, Kevin Carlson, Eswaran Subrahmanian, and Gioele Zardini. A categorical approach to semantic interoperability across building lifecycle, 2026.

[45] Ian Niles and Adam Pease. Towards a standard upper ontology. In *Proceedings of the International Conference on Formal Ontology in Information Systems (FOIS 2001)*, pages 2–9. ACM, 2001.

[46] Jacob Nilsson and Fredrik Sandin. Semantic interoperability in Industry 4.0: Survey of recent developments and outlook. In *2018 IEEE 16th International Conference on Industrial Informatics (INDIN)*, pages 2007–2012, 2018.

[47] Natalya F Noy. Semantic integration: a survey of ontology-based approaches. *ACM Sigmod Record*, 33(4):65–70, 2004.

[48] Mike P. Papazoglou and Dimitrios Georgakopoulos. Service-oriented computing. *Communications of the ACM*, 46(10):24–28, 2003.

[49] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77, 2007.

[50] Biren Prasad. *Concurrent engineering fundamentals*, volume 1. Prentice Hall PTR NJ, 1996.

[51] Raymond Reiter. On closed world data bases. In Hervé Gallaire and Jack Minker, editors, *Logic and Data Bases*, pages 55–76. Plenum Press, New York, NY, 1978.

[52] Mersedeh Sadeghi, Alessio Carenini, Oscar Corcho, Matteo Rossi, Riccardo Santoro, and Andreas Vogelsang. Interoperability of heterogeneous systems-of-systems: from requirements to a reference architecture. *The Journal of Supercomputing*, 80(7):8954–8987, 2024. Practitioner-grounded study of interoperability in loosely coupled SoS.

[53] Guilherme Santos Sousa, Rinaldo Lima, and Cássia Trojahn. Survey on embedding methods applied to ontology matching. *ACM Computing Surveys*, 58(12):1–40, 2026.

[54] Alexandra Schlemitz and Vitaliy Mezhuyev. Approaches for data collection and process standardization in smart manufacturing: Systematic literature review. *Journal of Industrial Information Integration*, 38:100578, 2024.

[55] G. Sohlenius. Concurrent engineering. *CIRP Annals*, 41(2):645–655, 1992.

[56] Josip Stjepandić, Nel Wognum, and Wim J.C. Verhagen, editors. *Concurrent Engineering in the 21st Century: Foundations, Developments and Challenges*. Springer, 2015.

[57] Maria Taboada, Diego Martínez, Mohammed Arideh, and Rosa Mosquera. Ontology matching with large language models and prioritized depth-first search. *Information Fusion*, 123:103254, 2025.

[58] Herman J. ter Horst. Completeness, decidability and complexity of entailment for RDF Schema and a semantic extension involving the OWL vocabulary. *Web Semantics: Science, Services and Agents on the World Wide Web*, 3(2–3):79–115, 2005.

[59] Andreas Tolk and James A. Muguira. The levels of conceptual interoperability model. In *Proceedings of the Fall Simulation Interoperability Workshop*, Orlando, FL, 2003.

[60] Jan van Deventer. A model based implementation of an iot framework. In *2025 IEEE International Symposium on Systems Engineering (ISSE)*, pages 1–7. IEEE, 2025.

[61] Gertjan van Heijst, A. Th. Schreiber, and B. J. Wielinga. Using explicit ontologies in KBS development. *International Journal of Human-Computer Studies*, 46(2-3):183–292, 1997.

[62] Pál Varga, Fredrik Blomstedt, Luis Lino Ferreira, Jens Eliasson, Mats Johansson, Jerker Delsing, and Iker Martínez de Soria. Making system of systems interoperable – the core components of the arrowhead framework. *Journal of Network and Computer Applications*, 81:85–95, 2017.

Department of Computer Science, Electrical and Space Engineering
Division of Embedded Intelligent Systems Lab

ISSN 1402-1544
ISBN 978-91-8142-117-0 (print)
ISBN 978-91-8142-118-7 (pdf)

Luleå University of Technology 2026

LULEÅ
UNIVERSITY
OF TECHNOLOGY